

考虑绿色限行与动态扰动的城市绿色物流配送调度优化研究

摘要

针对城市绿色物流配送中混合车队调度、软时间窗约束、时变交通、绿色配送区限行及动态订单扰动等问题,本文以总配送成本最小为目标,构建“静态调度-绿色限行-动态响应”的递进式优化框架。首先,对订单信息、距离矩阵、客户坐标和时间窗数据进行清洗匹配,统一客户编号、时间和距离口径;随后根据客户坐标识别绿色配送区,并对超容量客户进行任务化拆分,将 88 个有效客户转化为 237 个配送任务,为三类场景建立统一的任务集合和成本评价体系。

对于静态调度问题,本文建立考虑异质车队、载重与容积双容量约束、软时间窗、时变车速、能耗成本和碳排放成本的车辆路径优化模型,并采用多随机种子 ALNS 算法求解。无绿色限行时,最优方案启用车辆 140 辆,总配送距离为 18537.97 km,总配送成本为 115512.59 元。其中固定成本和能耗成本占比较高,迟到成本仅为 148.58 元,说明所得方案在控制运营成本的同时能够较好满足客户时效要求。

对于绿色限行调度问题,在静态模型基础上加入 8:00–16:00 燃油车禁入绿色配送区约束,并重新优化车辆分配、访问顺序和到达时刻。结果表明,限行后总成本增至 120037.28 元,较无政策情形增加 4524.70 元,增幅为 3.92%;总配送距离仅增加 94.98 km,说明成本上升主要并非由路径绕行导致,而是来源于等待成本和迟到成本增加。总碳排放量由 12146.42 kg 增至 12292.33 kg,表明在现有新能源运力有限的条件下,单纯实施绿色限行不一定降低企业整体碳排放。

对于动态事件调度问题,本文以问题二方案为基准,围绕订单取消、新增订单、配送地址变更和时间窗收紧四类事件,设计“状态冻结-任务更新-局部重优化”的滚动调度策略,并采用 LNS-VNS 算法对受影响路径及邻近路径进行低扰动修复。由于问题三仅评价事件发生后的剩余任务,本文以事件后剩余运营成本作为统一比较口径。四类事件重优化后均较初始响应方案降低成本,节约金额为 2504.43–3484.92 元,调整路径数控制在 4–6 条之间,说明该策略能够在降低剩余运营成本的同时有效控制路径扰动。

主要创新与特色:

1. **统一建模框架:** 以同一任务集合、车辆集合、约束体系和成本口径贯穿静态调度、绿色限行和动态响应三个问题,保证三类场景结果具有可比性和递进性。
2. **精细成本评价:** 将时变车速、载荷修正、油电能耗、碳排放成本、等待成本和迟到惩罚统一纳入路径评估器,能够逐弧计算车辆运行成本并解释成本变化来源。
3. **分场景算法适配:** 静态与绿色限行情形采用 ALNS 进行全局路径优化,动态事件采用 LNS-VNS 进行局部重优化,在求解质量、实时响应和路径稳定性之间取得平衡。

关键词: 绿色物流; 车辆路径优化; 软时间窗; ALNS; LNS-VNS

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题要求与研究思路	1
二、 模型假设	2
三、 符号说明	3
四、 数据处理与分析	4
4.1 数据整理与一致性检验	4
4.2 需求特征与绿色区识别	5
4.3 客户任务化处理	5
五、 问题一：静态调度模型建立与求解	6
5.1 问题分析	6
5.2 车辆路径调度模型的建立	7
5.3 模型求解	10
5.3.1 算法参数设置	11
5.4 结果分析	12
5.4.1 调度结果可行性与合理性验证	13
5.4.2 成本构成与碳排放分析	14
5.4.3 路径组织均衡性分析	15
六、 问题二：绿色限行调度模型建立与求解	15
6.1 问题分析	15
6.2 绿色配送区识别与限行约束建立	16
6.3 模型求解	16
6.4 调度结果分析与政策影响评价	17
七、 问题三：动态事件调度模型建立与求解	20
7.1 问题分析	20
7.2 动态调度模型建立	20
7.3 事件驱动滚动优化算法	22
7.3.1 动态重优化参数设置	22
7.4 动态事件案例设置与求解结果	24

7.5 结果分析	26
八、模型的优缺点分析	27
8.1 模型的优点	27
8.2 模型的不足	28
参考文献	28
A 附录 支撑材料文件列表	30
B 附录 代码	30

在此基础上，建立考虑异质车队、双容量约束、软时间窗、时变车速、能耗与碳排放成本的车辆路径优化模型，求解无政策限制下的基准调度方案。

2. **绿色限行政策下的调度重优化：**根据客户坐标识别绿色配送区客户，在基础模型中引入 8:00–16:00 燃油车禁入绿色配送区的政策约束，并采用 ALNS 算法重新优化车辆分配、访问顺序和到达时刻，进而分析限行政策对总成本、车辆使用结构、时间窗成本和碳排放水平的影响。

3. **动态事件下的滚动调度：**面向订单取消、新增订单、地址变更和时间窗调整等突发事件，构建“状态冻结–任务更新–局部重优化”的动态响应策略；在保留已执行方案的基础上，采用 LNS–VNS 算法修复受影响路径，评价动态扰动下的成本控制、路径扰动和方案稳定性。

二、模型假设

为保证模型具有可解性，并使建模口径与题目数据保持一致，本文作如下假设：

- **需求确定性假设：**客户配送需求由订单数据按客户编号汇总得到，在静态调度和绿色限行情形下视为已知确定量；除问题三所设动态事件外，不考虑其他随机需求扰动。
- **任务化服务假设：**对重量或体积需求超过单车容量上限的客户进行任务化拆分，拆分任务继承原客户的坐标、时间窗和绿色区属性。每个配送任务作为独立服务单元，仅由一辆车完成一次配送；对于同一客户拆分得到的多个任务，均按独立到达与卸货过程处理，服务时间仍取 20 min。
- **车辆运行假设：**所有车辆均从同一配送中心出发，并在完成所分配任务后返回配送中心；配送过程中不考虑临时换车、中途转运、途中补能及车辆故障等情形。
- **时间窗与交通假设：**时间以 8:00 为零点并统一换算为分钟。客户软时间窗作用于开始服务时刻，车辆可提前到达并等待，若开始服务时刻晚于时间窗上界则产生迟到惩罚。车辆行驶速度随交通时段变化，主模型采用各时段速度分布的期望值计算行驶时间，随机交通波动仅在鲁棒性检验中考虑。
- **能耗与排放假设：**车辆油耗或电耗由题目给定的车速–能耗函数确定，并按车辆载荷水平进行修正；能源成本与碳排放成本分别依据对应能源价格、碳排放转换系数和单位碳成本计算。
- **政策与动态调整假设：**绿色配送区依据客户坐标识别。由于题目未提供完整道路网络，本文以车辆到达绿色区客户点的时刻近似表示入区时刻。动态事件发生后，已完成服务及短时间内不可调整的车辆状态保持冻结，仅对后续可调整任务进行局部修复与滚动重优化。

三、符号说明

表 1 主要符号说明

符号	意义	单位
i, j, k, r	节点、车辆和车型索引，其中 0 表示配送中心	无量纲
C	有效客户集合，用于订单汇总、坐标识别和任务拆分	无量纲
N, V	配送任务集合与节点集合， $V = \{0\} \cup N$ ，其中 0 表示配送中心	无量纲
K, R, K_r	车辆集合、车型集合及车型 r 对应的车辆集合	无量纲
K^F, K^E, G	燃油车集合、新能源车集合与绿色区任务集合	无量纲
(x_i, y_i)	配送任务 i 继承的原客户二维坐标	km
q_i^w, q_i^v	配送任务 i 的重量需求与体积需求	kg; m ³
$[e_i, l_i], s_i$	配送任务 i 的软时间窗与服务时间	min
Q_k^w, Q_k^v	车辆 k 的最大载重与最大容积	kg; m ³
F_k, m_r	车辆固定启用成本与车型 r 的可用车辆数	元; 辆
d_{ij}, D	节点 i 到节点 j 的道路距离及距离矩阵	km
$\bar{v}(t), \tau_{ij}(t), \bar{v}_{ij}$	期望速度、时变行驶时间与弧 (i, j) 上的等效平均速度	km/h; min
$FPK(v)$	速度 v 下燃油车百公里油耗	L/100 km
$EPK(v)$	速度 v 下新能源车百公里电耗	kWh/100 km
$R_{ijk}^w, R_{ijk}^v, \rho_{ijk}$	车辆 k 在弧 (i, j) 出发时的剩余载荷与综合载荷率	kg; m ³ ; 无量纲
ϕ_F, ϕ_E	燃油车与新能源车的载荷修正因子	无量纲
p_f	单位油价	元/L
p_e	单位电价	元/kWh
η	油耗碳排放转换系数	kg/L
γ	电耗碳排放转换系数	kg/kWh

续表 1 主要符号说明

符号	意义	单位
M	大常数	无量纲
x_{ijk}	若车辆 k 从节点 i 行驶至节点 j , 则取 1, 否则取 0	无量纲
g_{ik}, z_k	任务分配变量与车辆启用变量	无量纲
A_{ik}, B_{ik}	车辆 k 到达配送任务 i 的时刻与开始服务时刻	min
W_{ik}, L_{ik}	车辆 k 在配送任务 i 处的等待时间与迟到时间	min
$T_k^{\text{dep}}, T_k^{\text{ret}}$	车辆 k 离开与返回配送中心的时刻	min
U_{ik}^w, U_{ik}^v	车辆 k 完成配送任务 i 服务后的累计配送重量与体积	kg; m ³
Z	总配送成本	元
$c_{ijk}^{\text{ene}}, c_{ijk}^{\text{car}}$	车辆 k 行驶弧 (i, j) 产生的能源成本与碳排放成本	元
$\hat{C}_{\text{ene}}, \hat{C}_{\text{car}}$	路径评估器累计得到的总能源成本与总碳排放成本	元
S, S', S^*	当前解、候选解与当前最优解	无量纲
$N^{\text{done}}, N^{\text{lock}}$	已完成任务集合与暂不可调整任务集合	无量纲
$N^{\text{rem}}, N^{\text{new}}$	尚未服务任务集合与新增订单任务集合	无量纲
$\tilde{N}^{\text{rem}}, N^{\text{dyn}}$	事件更新后的剩余任务集合与动态待优化任务集合	无量纲
$R^0, R^{\text{init}}, R^*$	基准、初始响应与动态重优化方案	无量纲
$K^{\text{aff}}, K^{\text{nei}}$	受影响车辆集合与邻近车辆集合	无量纲
K^{sub}	动态优化子问题车辆集合	无量纲
T_k^{ava}	车辆 k 在动态事件后的最早可用时刻	min
$Z^{\text{dyn}}, C_{\text{oper}}^{\text{dyn}}$	动态调度目标函数值与事件后剩余任务运营成本	元

四、数据处理与分析

4.1 数据整理与一致性检验

题目附件包括订单信息、距离矩阵、客户坐标信息和时间窗信息四类数据。本文以客户编号为主键, 对订单数据进行汇总, 得到各客户的总需求重量与总体积; 随后与客户坐标、时间窗和距离矩阵进行匹配, 构建统一客户数据表, 作为后续建模与算法求解

的基础输入。

数据检验结果表明，坐标表和时间窗表共包含 98 个客户点，其中订单数据实际覆盖 88 个客户，即有 10 个客户当日无配送需求。因此，本文将 98 个客户点作为全集客户，用于空间分布分析和绿色区识别；将 88 个有订单需求的客户作为有效客户，用于车辆路径规划。清洗与匹配后，各关键字段均无缺失，数据一致性满足建模要求。

为统一时间计算口径，本文将时间窗转换为以 8:00 为零点的分钟数。根据题目补充说明，顺畅、一般和拥堵时段的车辆速度分别服从

$$N(55.3, 0.1^2), \quad N(35.4, 5.2^2), \quad N(9.8, 4.7^2).$$

主模型求解时采用各时段速度均值计算行驶时间，并在鲁棒性分析中基于上述分布进行随机扰动。

4.2 需求特征与绿色区识别

绿色配送区为以市中心为圆心、半径 10km 的圆形区域。根据题目补充说明，绿色区客户数量应由坐标数据计算确定。设客户 i 的坐标为 (x_i, y_i) ，则绿色区客户判定条件为

$$x_i^2 + y_i^2 \leq 10^2.$$

计算结果表明，98 个全集客户中共有 15 个位于绿色配送区；在 88 个有效客户中，共有 12 个绿色区客户。因此，问题二中的绿色限行约束主要作用于这些有效绿色客户及其拆分后的虚拟配送任务。虽然绿色区客户占比不高，但燃油车在 8:00–16:00 期间无法进入该区域，会对车辆分配、到达时刻和路径重组产生直接影响。

从需求分布看，客户重量需求和体积需求均呈现右偏特征，即多数客户需求较小，少数客户需求显著偏大。因此，车辆调度不能采用简单平均分配策略，而需综合考虑车型差异、载重约束、容积约束和客户空间位置。

4.3 客户任务化处理

由于部分客户的汇总需求超过单车载重或容积限制，若将其视为单一服务对象，可能导致路径构造不可行。为此，本文对超容量客户进行任务化拆分，将其转化为若干个满足单车容量约束的虚拟配送任务。拆分任务继承原客户的坐标、时间窗和绿色区属性，并作为后续路径优化的基本服务单元。

设有效客户集合为 C 。对于原客户 $c \in C$ ，其订单汇总后的重量需求和体积需求分别记为 $\bar{q}_c^w$ 和 $\bar{q}_c^v$ 。若客户 c 的需求超过单车容量上限，则将其拆分为 h_c 个虚拟配送任务，记为 $c_1, c_2, \dots, c_{h_c}$ 。为保证需求守恒，拆分结果满足

$$\sum_{\ell=1}^{h_c} q_{c\ell}^w = \bar{q}_c^w, \quad \sum_{\ell=1}^{h_c} q_{c\ell}^v = \bar{q}_c^v, \quad (1)$$

其中, q_{cl}^w 和 q_{cl}^v 分别表示客户 c 拆分所得第 ℓ 个配送任务的重量和体积需求。

经任务化处理后, 88 个有效客户被转化为 237 个配送任务。记任务化后的配送任务集合为 $N = \{1, 2, \dots, n\}$, 其中 $n = 237$ 。每个配送任务继承其原客户的坐标、时间窗和绿色区属性, 并作为后续车辆路径优化的基本服务单元。对于同一客户拆分得到的多个配送任务, 本文均视为独立车辆到达与卸货过程, 并统一设置服务时间 $s_i = 20 \text{ min}$, 以避免低估服务时间和时间窗压力。

五、问题一：静态调度模型建立与求解

5.1 问题分析

问题一是在无绿色配送区限行政策条件下的基础静态调度问题。其核心是在完成全部配送任务的前提下, 综合确定车辆启用、车型选择、任务分配、任务访问顺序和到达服务时刻, 使固定启用成本、能源成本、碳排放成本、等待成本和迟到惩罚成本之和最小。

与经典车辆路径问题^[1]相比, 本题约束耦合更强: 车队包含燃油车和新能源车, 属于混合车队调度问题^[2]; 配送任务需求同时受重量和体积双容量约束; 软时间窗对应早到等待和晚到惩罚^[3]; 时变车速则使访问顺序、出发时刻、能耗和成本相互影响^[4,5]。

因此, 本文将问题一归结为带异质车队、双容量约束、软时间窗和时变速度的车辆路径优化问题。模型以总配送成本最小为目标, 在任务唯一服务、车辆容量、路径连续、时间递推和车队数量限制等约束下求解基础调度方案, 并为问题二的绿色限行扩展和问题三的动态重优化提供基准方案。



图 2 问题一求解流程图

5.2 车辆路径调度模型的建立

为刻画混合车队在时变交通条件下的配送成本, 本文借鉴污染路径问题和时变污染路径问题的建模思想^[6,7], 构建带软时间窗和双容量约束的异质车辆路径优化模型。模型以总配送成本最小为目标, 综合决定车辆启用、任务分配、任务访问顺序以及各节点到达与服务时刻。

设配送中心为节点 0, 任务化处理后的配送任务集合为 $N = \{1, 2, \dots, n\}$, 其中 $n = 237$, 节点集合为 $V = \{0\} \cup N$ 。若配送任务 i 由某一原客户拆分得到, 则其坐标、时间窗和绿色区属性继承该原客户。客户任务化后, 模型中的节点 $i \in N$ 均表示一个配送任务, 而非原始客户点。车辆集合为 K , 其中 K^F 和 K^E 分别表示燃油车集合与新能源车集合, K_r 表示车型 r 对应的车辆集合。配送任务 i 的需求、时间窗和服务时间分别为 $q_i^w, q_i^v, [e_i, l_i]$ 和 s_i , 车辆 k 的载重、容积和固定启用成本分别为 Q_k^w, Q_k^v 和 F_k , 节点间道路距离为 d_{ij} 。

定义 x_{ijk} 表示车辆 k 是否行驶弧 (i, j) , g_{ik} 表示配送任务 i 是否由车辆 k 服务, z_k 表示车辆 k 是否被启用; A_{ik} 、 B_{ik} 、 W_{ik} 和 L_{ik} 分别表示车辆到达时刻、开始服务时刻、等待时间和迟到时间; T_k^{dep} 和 T_k^{ret} 分别表示离开与返回配送中心时刻; U_{ik}^w 和 U_{ik}^v 表示完成配送任务 i 服务后的累计配送重量与体积。二元变量定义为

$$g_{ik} = \begin{cases} 1, & \text{配送任务 } i \text{ 由车辆 } k \text{ 服务,} \\ 0, & \text{否则,} \end{cases} \quad z_k = \begin{cases} 1, & \text{车辆 } k \text{ 被启用,} \\ 0, & \text{否则.} \end{cases} \quad (2)$$

考虑到车辆速度随交通时段变化, 本文以 8:00 为时间零点, 并采用题目给定速度分布的期望值构造确定性时变速度函数: (由于题目未给出 17:00 之后的速度分布, 本文延用一般时段期望速度 35.4 km/h 作为后续时段速度。)

$$\bar{v}(t) = \begin{cases} 9.8, & t \in [0, 60) \cup [210, 300), \\ 55.3, & t \in [60, 120) \cup [300, 420), \\ 35.4, & t \in [120, 210) \cup [420, +\infty). \end{cases} \quad (3)$$

若车辆在时刻 t 从节点 i 出发前往节点 j , 则行驶时间定义为

$$\tau_{ij}(t) = \inf \left\{ \Delta \geq 0 : \int_t^{t+\Delta} \frac{\bar{v}(u)}{60} du \geq d_{ij} \right\}. \quad (4)$$

其中, $\bar{v}(u)$ 的单位为 km/h, 除以 60 后转化为 km/min。若行驶过程中跨越多个交通时段, 则按分段速度累计行驶距离, 直至覆盖弧段距离 d_{ij} , 以更准确刻画时变交通对到达时刻的影响^[5]。

题目给出的燃油车百公里油耗和新能源车百公里电耗函数分别为

$$FPK(v) = 0.0025v^2 - 0.2554v + 31.75, \quad EPK(v) = 0.001v^2 - 0.1v + 36.194. \quad (5)$$

由于车辆能耗不仅与车速有关，还受载荷水平影响，本文在路径评估器中对每条行驶弧进行逐弧计算。定义车辆 k 在弧 (i, j) 出发时的综合载荷率为

$$\rho_{ijk} = \max \left\{ \frac{R_{ijk}^w}{Q_k^w}, \frac{R_{ijk}^v}{Q_k^v} \right\}, \quad 0 \leq \rho_{ijk} \leq 1, \quad (6)$$

其中， R_{ijk}^w 和 R_{ijk}^v 分别表示车辆在弧 (i, j) 出发时的剩余重量载荷和剩余体积载荷。根据题意，燃油车满载能耗较空载提高 40%，新能源车满载能耗较空载提高 35%，故设载荷修正因子为

$$\phi_F(\rho_{ijk}) = 1 + 0.40\rho_{ijk}, \quad \phi_E(\rho_{ijk}) = 1 + 0.35\rho_{ijk}. \quad (7)$$

在给定车辆路径后，路径评估器根据车辆类型、出发时刻、弧段距离和载荷状态逐弧计算能源成本与碳排放成本。设 $\bar{v}_{ij} = 60d_{ij}/\tau_{ij}(t)$ 为车辆在时刻 t 从节点 i 出发驶向节点 j 时对应的等效平均速度，则单弧能源成本和碳排放成本分别为

$$\begin{aligned} c_{ijk}^{\text{ene}} &= \begin{cases} \frac{d_{ij}}{100} FPK(\bar{v}_{ij}) \phi_F(\rho_{ijk}) p_f, & k \in K^F, \\ \frac{d_{ij}}{100} EPK(\bar{v}_{ij}) \phi_E(\rho_{ijk}) p_e, & k \in K^E, \end{cases} \\ c_{ijk}^{\text{car}} &= \begin{cases} \frac{d_{ij}}{100} FPK(\bar{v}_{ij}) \phi_F(\rho_{ijk}) \eta c_c, & k \in K^F, \\ \frac{d_{ij}}{100} EPK(\bar{v}_{ij}) \phi_E(\rho_{ijk}) \gamma c_c, & k \in K^E. \end{cases} \end{aligned} \quad (8)$$

其中， p_f 和 p_e 分别为油价和电价， η 和 γ 分别为燃油与电耗的碳排放转换系数， c_c 为单位碳排放成本。

需要说明的是，能源成本和碳排放成本同时依赖于车辆类型、出发时刻、路径顺序和载荷变化，难以简单表示为固定弧成本。因此，本文采用“主模型刻画可行域、路径评估器计算行驶成本”的处理方式，将路径评估器逐弧累计得到的能源成本和碳排放成本分别记为 $\hat{C}_{\text{ene}}(x, B, U)$ 和 $\hat{C}_{\text{car}}(x, B, U)$ 。于是，问题一的目标函数为

$$\min Z = \sum_{k \in K} F_k z_k + \hat{C}_{\text{ene}}(x, B, U) + \hat{C}_{\text{car}}(x, B, U) + c_w \sum_{k \in K} \sum_{i \in N} W_{ik} + c_l \sum_{k \in K} \sum_{i \in N} L_{ik}. \quad (9)$$

其中，第一项为车辆固定启用成本，第二、三项分别为能源成本和碳排放成本，第四、五项分别为等待成本和迟到惩罚成本。

模型约束如下。

首先，每个配送任务必须且只能被一辆车服务，且任务服务变量与路径变量保持一致：

$$\begin{cases} \sum_{k \in K} g_{ik} = 1, & \forall i \in N, \\ \sum_{\substack{j \in V \\ j \neq i}} x_{ijk} = g_{ik}, & \forall i \in N, k \in K, \\ \sum_{\substack{j \in V \\ j \neq i}} x_{jik} = g_{ik}, & \forall i \in N, k \in K. \end{cases} \quad (10)$$

车辆若被启用，则必须从配送中心出发并最终返回配送中心；未启用车辆不能服务配送任务：

$$\begin{cases} \sum_{j \in N} x_{0jk} = z_k, & \forall k \in K, \\ \sum_{i \in N} x_{i0k} = z_k, & \forall k \in K, \\ g_{ik} \leq z_k, & \forall i \in N, k \in K. \end{cases} \quad (11)$$

车辆载重和容积均不能超过对应车型容量：

$$\begin{cases} \sum_{i \in N} q_i^w g_{ik} \leq Q_k^w z_k, \\ \sum_{i \in N} q_i^v g_{ik} \leq Q_k^v z_k, \end{cases} \quad \forall k \in K. \quad (12)$$

为防止不经过配送中心的孤立子回路，引入累计配送量约束：

$$\begin{cases} q_i^w g_{ik} \leq U_{ik}^w \leq Q_k^w g_{ik}, & \forall i \in N, k \in K, \\ U_{jk}^w \geq U_{ik}^w + q_j^w - Q_k^w (1 - x_{ijk}), & \forall i, j \in N, i \neq j, k \in K, \\ q_i^v g_{ik} \leq U_{ik}^v \leq Q_k^v g_{ik}, & \forall i \in N, k \in K, \\ U_{jk}^v \geq U_{ik}^v + q_j^v - Q_k^v (1 - x_{ijk}), & \forall i, j \in N, i \neq j, k \in K. \end{cases} \quad (13)$$

车辆时间递推关系如下：

$$\begin{cases} 0 \leq T_k^{\text{dep}} \leq T_k^{\text{ret}} \leq M z_k, & \forall k \in K, \\ A_{jk} \geq T_k^{\text{dep}} + \tau_{0j}(T_k^{\text{dep}}) - M(1 - x_{0jk}), & \forall j \in N, k \in K, \\ A_{jk} \geq B_{ik} + s_i + \tau_{ij}(B_{ik} + s_i) - M(1 - x_{ijk}), & \forall i, j \in N, i \neq j, k \in K, \\ T_k^{\text{ret}} \geq B_{ik} + s_i + \tau_{i0}(B_{ik} + s_i) - M(1 - x_{i0k}), & \forall i \in N, k \in K. \end{cases} \quad (14)$$

软时间窗作用于配送任务的开始服务时刻。车辆可以提前到达并等待，若开始服务时刻晚于时间窗上界，则产生迟到惩罚：

$$\begin{cases} B_{ik} \geq A_{ik} - M(1 - g_{ik}), & \forall i \in N, k \in K, \\ B_{ik} \geq e_i - M(1 - g_{ik}), & \forall i \in N, k \in K, \\ W_{ik} \geq B_{ik} - A_{ik} - M(1 - g_{ik}), & \forall i \in N, k \in K, \\ L_{ik} \geq B_{ik} - l_i - M(1 - g_{ik}), & \forall i \in N, k \in K, \\ 0 \leq W_{ik} \leq M g_{ik}, \quad 0 \leq L_{ik} \leq M g_{ik}, & \forall i \in N, k \in K. \end{cases} \quad (15)$$

每种车型的启用数量不能超过题目给定车队规模：

$$\sum_{k \in K_r} z_k \leq m_r, \quad \forall r \in R. \quad (16)$$

最后，变量取值约束为

$$\begin{cases} x_{ijk}, g_{ik}, z_k \in \{0, 1\}, & \forall i, j \in V, i \neq j, k \in K, \\ A_{ik}, B_{ik}, W_{ik}, L_{ik}, U_{ik}^w, U_{ik}^v \in \mathbb{R}_+, & \forall i \in N, k \in K, \\ T_k^{\text{dep}}, T_k^{\text{ret}} \in \mathbb{R}_+, & \forall k \in K. \end{cases} \quad (17)$$

综上，式 (9) 为总配送成本最小化目标函数，式 (10)–(17) 分别刻画任务服务与流平衡、车辆启用、双容量限制、子回路消除、时间递推、软时间窗、车型数量及变量取值约束，由此完成问题一静态车辆路径调度模型的建立。

5.3 模型求解

问题一同时涉及异质车队选择、任务分配、路径排序、双容量约束、软时间窗约束以及时变速度下的成本评估，属于约束耦合较强的组合优化问题。若采用精确算法直接求解，变量规模和约束数量较大，难以满足实际调度效率要求。因此，本文借鉴 Shaw 提出的大邻域搜索思想^[8]，并参考 Ropke 和 Pisinger 提出的自适应大邻域搜索框架^[9]，设计基于 ALNS 的多算子大邻域搜索算法进行求解。

算法采用“初始解构造–破坏修复–路径评估–概率接受–权重更新–多种子择优”的框架。先根据需求规模、时间窗紧迫度和可服务车型构造任务优先级，并用贪婪插入生成初始可行解；再通过破坏修复调整任务分配与访问顺序，并调用路径评估器计算时刻和成本。候选解按模拟退火准则接受^[10]，算子权重随搜索表现动态更新。流程见算法1。

Algorithm 1 基于 ALNS 的车辆配送路径优化算法

输入： 配送任务集合 N ，车辆集合 K ，距离矩阵 D ，任务需求 q_i^w, q_i^v ，时间窗 $[e_i, l_i]$ ，随机种子集合 $\mathcal{S}$ ，最大迭代次数 $I_{\max}$ ，运行时间上限 $t_{\max}$

输出： 最优配送方案 S^* 及其总成本 $Z(S^*)$

```
1: 对订单、时间窗、距离矩阵和车辆参数进行预处理；
2: 构建路径评估器  $\text{Eval}(\cdot)$ ，设定破坏算子集合  $\mathcal{D}$  与修复算子集合  $\mathcal{R}$ ；
3: 初始化全局最优解  $S^* \leftarrow \emptyset$ ， $Z(S^*) \leftarrow +\infty$ ；
4: for 每个随机种子  $s \in \mathcal{S}$  do
5:   设置随机种子  $s$ ，采用贪婪插入策略生成初始可行解  $S$ ；
6:   初始化破坏算子和修复算子的选择权重；
7:   令当前种子最优解  $S^{best} \leftarrow S$ ，温度  $T \leftarrow T_0$ ；
8:   for  $iter = 1$  to  $I_{\max}$  且运行时间未超过  $t_{\max}$  do
9:     按当前权重从  $\mathcal{D}$  和  $\mathcal{R}$  中选择破坏算子  $d$  与修复算子  $r$ ；
10:    使用  $d$  从当前解  $S$  中移除部分任务，得到部分解  $\bar{S}$ ；
11:    使用  $r$  将被移除任务重新插入  $\bar{S}$ ，生成候选解  $S'$ ；
12:    调用  $\text{Eval}(S')$  计算成本，并检验容量、时间窗、车型数量及相应场景约束；
13:    if  $S'$  可行 then
14:       $\Delta Z \leftarrow Z(S') - Z(S)$ ；
15:      if  $\Delta Z < 0$  或以概率  $\exp(-\Delta Z/T)$  接受 then
16:         $S \leftarrow S'$ ；
17:      end if
18:      if  $Z(S) < Z(S^{best})$  then
19:         $S^{best} \leftarrow S$ ；
20:      end if
21:    end if
22:    根据候选解表现更新破坏算子与修复算子的选择权重；
23:    按  $T \leftarrow \alpha T$  更新温度；
24:  end for
25:  if  $Z(S^{best}) < Z(S^*)$  then
26:     $S^* \leftarrow S^{best}$ ；
27:  end if
28: end for
29: return  $S^*$ 。
```

破坏阶段设置随机移除、最差任务移除、相关任务移除和整条路径移除，用于增强扰动、调整高成本任务、重构相近任务群和优化车辆启用结构；修复阶段采用最小增量成本插入、regret 插入和时间窗优先插入，将任务重新插入可行路径。各算子初始权重相同，迭代中按产生优质解或可接受改进解的表现获得奖励，并据此更新选择权重。

为降低初始解构造和随机算子选择对结果的影响，本文采用多随机种子独立运行策略。对不同随机种子得到的候选解进行统一可行性检验，并在所有可行解中选取总配送成本最低的方案作为问题一最终结果。最终输出车辆使用方案、配送路径、到达与开始服务时刻，以及固定成本、能源成本、碳排放成本、等待成本和迟到惩罚成本等指标。

5.3.1 算法参数设置

为保证实验结果的可复现性，本文统一设定问题一 ALNS 算法的主要参数，见表 2。算法采用多随机种子独立运行，并在通过任务覆盖、车型数量、载重容量、容积容量和软时间窗计罚规则检验的可行方案中，选取总配送成本最低者作为最终调度结果。

表 2 问题一 ALNS 求解参数设置

参数	含义	取值
S	多随机种子集合	$\{1, 7, 21, 42, 66, 88, 100\}$
$I_{\max}$	最大迭代次数	300
$t_{\max}$	单个随机种子的运行时间上限	180 s
T_0	模拟退火初始温度	2000
α	降温系数, 按 $T \leftarrow \alpha T$ 更新	0.985
p	每次破坏操作移除的任务比例	$[0.04, 0.16]$
c_w	单位等待成本	20/60 元/min
c_l	单位迟到惩罚成本	50/60 元/min
c_c	单位碳排放成本	0.65 元/kg

在候选解接受阶段, 设当前解和候选解分别为 S 与 S' 。若 $Z(S') < Z(S)$, 则直接接受候选解; 否则以概率

$$P = \exp\left(-\frac{Z(S') - Z(S)}{T}\right)$$

接受该候选解, 并在每轮迭代后按 $T \leftarrow \alpha T$ 更新温度。该机制使算法在搜索初期保留一定的全局探索能力, 后期逐步转向稳定收敛。

问题二在相同求解框架下进一步嵌入绿色配送区限行检验: 若燃油车在 8:00–16:00 到达绿色区任务, 则该插入方案判定为不可行。每次候选解生成后, 均调用路径评估器重新计算时间可行性与成本构成, 以保证算法评价口径与目标函数一致。

5.4 结果分析

基于上述参数设置, 本文对问题一进行多随机种子求解, 并统计各次运行的成本水平与可行性, 结果如表 3 所示。

表 3 问题一 ALNS 多随机种子求解稳定性验证

种子数	最优种子	最优成本/元	平均成本/元	变异系数/%	可行率/%
7	100	115512.59	116559.92	0.613	100

由表 3 可知，问题一在 7 个随机种子下均得到可行解，可行率为 100%；目标函数值的变异系数为 0.613%，说明算法结果对随机种子的敏感性较低，具有较好的求解稳定性。其中，随机种子为 100 时取得最低总成本 115512.59 元，故本文选取该方案作为问题一最终调度方案。

问题一最终优化结果如表 4 所示。

表 4 问题一最终优化结果汇总

指标	数值	指标	数值
最优随机种子	100	配送任务数	237
使用车辆数	140	总配送距离/km	18537.97
总成本/元	115512.59	固定成本/元	56000.00
能耗成本/元	36526.07	碳排放成本/元	7895.17
等待成本/元	14942.76	迟到成本/元	148.58
任务覆盖	全部覆盖	车型数量	满足约束

由表 4 可知，问题一最优方案共启用车辆 140 辆，完成 237 个配送任务，总配送距离为 18537.97 km，总成本为 115512.59 元。其中，固定成本为 56000.00 元，能耗成本为 36526.07 元，二者是总成本的主要来源；等待成本为 14942.76 元，说明软时间窗对路径时序安排具有明显影响；迟到成本仅为 148.58 元，表明方案能够较好地控制服务延误。

5.4.1 调度结果可行性与合理性验证

本文采用贪婪插入策略构造初始可行解，并通过 ALNS 的破坏–修复机制持续改进。最终方案通过了任务覆盖、车型数量和时间窗成本三类检验：一是 237 个配送任务全部被覆盖，未出现漏送或重复服务；二是各车型启用数量均未超过题目给定车队规模；三是迟到成本仅占总成本的 0.13%，说明算法在降低总成本的同时较好地控制了服务延误。

从车型启用结构看，方案呈现“大容量车辆主导、新能源车辆充分利用、小型车辆补充服务”的特征。燃油车 3000 kg、1500 kg 和 1250 kg 型分别启用 60、49 和 6 辆；新能源车 3000 kg 和 1250 kg 型分别启用 10 和 15 辆，均被全部启用。

为进一步说明调度方案的可执行性，本文选取部分代表性车辆路径，列出车辆访问顺序、任务到达时刻和开始服务时刻，如表 5 所示。其中，任务节点括号内的两个时间分别表示车辆到达该配送任务的时刻和开始服务时刻。

表 5 问题一部分车辆路径与到达时间示例

路径编号	车型	访问顺序	各任务到达 → 开始服务时刻	距离/km	成本/元
R003	燃油 3000	0 → 32_1 → 75_2 → 45_1 → 0	32_1: 10:42 → 13:08; 75_2: 13:47 → 14:25; 45_1: 15:18 → 15:37	174.47	993.29
R013	新能源 3000	0 → 87_1 → 39_3 → 57_4 → 0	87_1: 10:49 → 11:18; 39_3: 13:07 → 14:08; 57_4: 16:07 → 16:07	213.21	611.02
R016	新能源 1250	0 → 74_1 → 0	74_1: 10:01 → 13:11	131.67	566.19
R041	燃油 1500	0 → 48_2 → 0	48_2: 09:28 → 16:06	71.56	742.05

注：0 表示配送中心；32_1 表示客户 32 拆分得到的第 1 个配送任务；“到达 → 开始服务”表示车辆到达配送任务对应客户点后，若早于时间窗下界，则需等待至允许服务时刻后开始服务。

由表 5 可见，问题一调度方案能够同时给出车辆访问顺序、任务到达时刻和开始服务时刻。例如，路径 R003 中车辆于 10:42 到达任务 32_1，但由于早于该任务允许服务时刻，需等待至 13:08 后开始服务；路径 R013 中任务 57_4 的到达时刻与开始服务时刻均为 16:07，说明该节点无需等待即可服务。由此可见，路径评估器能够正确刻画提前到达等待和按时间窗开始服务的过程。

5.4.2 成本构成与碳排放分析

问题一成本构成如图 3 所示。

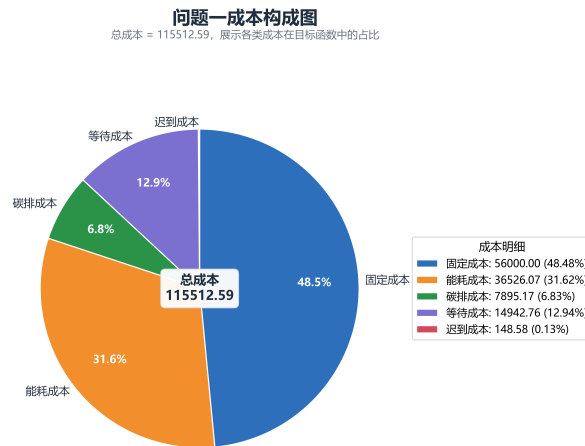


图 3 问题一成本构成图

由图 3 可知，固定成本和能耗成本分别占 48.48% 和 31.62%，合计超过 80%，是静态调度成本的主要来源。等待成本占 12.94%，碳排放成本占 6.83%，迟到成本仅占 0.13%。

进一步地，根据碳排放成本和单位碳成本可估算问题一方案的总碳排放量。由于单位碳排放成本为 0.65 元/kg，因此有

$$E_1 = \frac{7895.17}{0.65} = 12146.42 \text{ kg.}$$

该结果作为问题二分析绿色配送区限行政策对碳排放影响的基准值。

5.4.3 路径组织均衡性分析

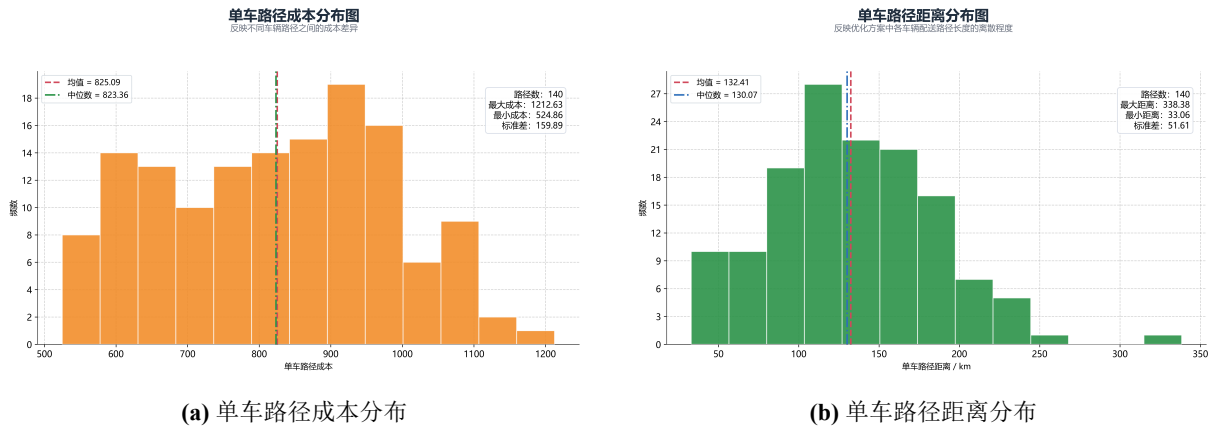


图 4 问题一单车路径成本与距离分布

为考察路径组织均衡性，本文统计单车路径成本与距离，结果如图 4 所示。单车成本主要集中在 600–1000 元，平均值 825.09 元，中位数 823.36 元；单车距离主要集中在 100–200 km，平均值 132.41 km，中位数 130.07 km，说明路径规模较集中。

此外，问题一方案的平均单车服务任务数为

$$\frac{237}{140} = 1.69.$$

该数值相对较小，主要是由于任务化拆分后，部分配送任务仍受到载重、容积、时间窗和空间分布等多重约束限制，难以进一步合并。因此，较高的车辆启用数量并非单纯由算法搜索不足造成，而是多重约束共同作用下形成的可行调度结果。

综上，问题一所得方案能够完成全部配送任务，并满足容量、路径连续性、车型数量和软时间窗计罚要求。结果表明，固定成本和能耗成本是总成本的主要来源，软时间窗约束对路径时序安排具有明显影响，而迟到成本较低，说明所设计算法能够有效控制配送延误。该方案可作为问题二绿色限行扩展和问题三动态重优化的基准方案。

六、问题二：绿色限行调度模型建立与求解

6.1 问题分析

问题二是在问题一静态调度模型基础上加入绿色配送区限行政策后的再优化问题。根据题意，8:00–16:00 期间燃油车禁止进入以市中心为圆心、半径 10 km 的绿色配送区，因此问题一所得方案在该政策下可能不再可行，需要重新调整车辆分配、任务访问顺序和到达时刻。

与问题一相比，问题二的配送任务需求、车辆容量、时间窗和成本构成均保持不变，主要变化在于燃油车对绿色区任务的服务时段受到限制，使可行解空间进一步压缩。因

此，问题二可归结为带绿色配送区限行约束的异质车队车辆路径优化问题，重点是在新能源车辆优先服务、燃油车辆延后服务和路径重组之间进行权衡，并评估限行政策对总成本、车辆结构和碳排放水平的影响^[2,6,7]。

6.2 绿色配送区识别与限行约束建立

在问题一模型基础上，问题二进一步识别绿色配送区客户，并将其拆分后得到的配送任务纳入绿色区任务集合。由于配送任务继承原客户坐标，绿色区任务集合可表示为

$$G = \{i \in N \mid x_i^2 + y_i^2 \leq 10^2\}. \quad (18)$$

对于 $i \in G$ 的配送任务，新能源车辆可在任意时段进入并完成配送；若由燃油车服务，则其到达时刻不得早于 16:00。以 8:00 为时间零点，16:00 对应 480 min。沿用问题一中的任务分配变量 g_{ik} 和到达时刻变量 A_{ik} ，对任意 $i \in G$ 、 $k \in K^F$ ，建立限行约束：

$$A_{ik} \geq 480 - M(1 - g_{ik}), \quad i \in G, k \in K^F, \quad (19)$$

其中， M 为充分大的正常数。当 $g_{ik} = 1$ 时，式 (19) 退化为 $A_{ik} \geq 480$ ，表示燃油车只能在 16:00 后到达绿色区任务；当 $g_{ik} = 0$ 时，该约束自动松弛。

需要说明的是，题目附件仅提供客户坐标和点间道路距离矩阵，未给出完整道路网络及车辆穿越绿色区边界的具体位置。因此，本文以车辆到达绿色区任务所对应客户点的时刻近似表示进入绿色配送区的时刻。该处理与数据粒度相匹配，能够保证绿色区任务在限行时段内不由燃油车直接服务。若考虑更保守的调度情形，可引入安全缓冲时间 δ ，将约束改写为

$$A_{ik} \geq 480 + \delta - M(1 - g_{ik}), \quad i \in G, k \in K^F.$$

本文在缺少完整路网数据的条件下取 $\delta = 0$ 。

6.3 模型求解

问题二在问题一 ALNS 求解框架的基础上进行扩展，仍采用“初始解构造-破坏修复-路径评估-概率接受-多随机种子择优”的搜索流程^[8,9]。与问题一相比，问题二的配送任务需求、车辆容量、时间窗和成本评价口径保持不变，主要区别在于算法在初始解构造、任务重插入和候选解可行性检验中嵌入绿色配送区限行规则。

具体而言，对于绿色区任务，算法在插入阶段优先尝试将其分配至新能源车辆路径；若候选插入方案由燃油车服务绿色区任务，则进一步检查其到达时刻。若燃油车到达绿色区任务的时刻早于 16:00，则该插入位置被判定为不可行；仅当其到达时刻不早于 16:00 时，才允许进入候选解。通过该方式，绿色限行约束被直接嵌入 ALNS 的可行性检验过程，从而避免生成违反政策约束的路径方案。

为降低随机初始化和邻域算子选择对结果的影响，本文仍采用多随机种子独立运行策略，随机种子集合设为

$$\mathcal{S} = \{1, 7, 21, 42, 66, 88, 100\}.$$

各随机种子下独立执行 ALNS 搜索，并在所有满足容量、时间窗、车型数量和绿色限行约束的可行方案中，选取总成本最低者作为问题二最终调度结果。

为保证问题一与问题二结果具有可比性，问题二沿用问题一的随机种子集合、最大迭代次数、运行时间上限、降温系数和成本评价口径。考虑到绿色限行约束会压缩可行解空间，本文适当提高模拟退火初始温度并扩大破坏比例，以增强算法跳出局部可行域的能力。参数调整如表 6 所示，其余参数与问题一保持一致。

表 6 问题二 ALNS 参数调整说明

参数	问题一取值	问题二取值
T_0	2000	2500
p	[0.04, 0.16]	[0.04, 0.18]

为验证限行约束的执行情况，本文对最终方案进行绿色配送区合规审计，统计绿色区服务节点数、燃油车服务绿色区节点数和限行违规节点数。审计结果表明，最终方案中燃油车在 8:00–16:00 期间服务绿色区任务的违规节点数为 0，说明所得方案满足绿色配送区限行政策要求。

6.4 调度结果分析与政策影响评价

问题二仍采用 7 个随机种子独立求解，所有结果均满足容量、时间窗、车型数量和绿色限行约束。其中，随机种子 66 取得最低总成本 120037.28 元，故作为最终方案。该方案启用车辆 141 辆，总距离 18632.96 km，碳排放量 12292.33 kg；燃油车 3000 kg、1500 kg、1250 kg 型分别启用 60、49、7 辆，新能源车 3000 kg、1250 kg 型分别启用 10、15 辆。

为检验绿色限行约束，本文选取部分代表性路径，列出访问顺序、到达与开始服务时刻及限行核验结果，见表 7。其中，“到达 → 开始服务”表示车辆早于时间窗下界到达时需等待至允许服务时刻。

表 7 问题二绿色限行下部分车辆路径与到达时间示例

路径编号	车型	访问顺序	各任务到达 → 开始服务时刻	限行核验	距离/km	成本/元
R001	燃油 1500	0 → 6_2 → 0	6_2: 16:00 → 16:00	16:00 到达绿色区任务, 满足限行约束	81.28	917.96
R012	新能源 3000	0 → 88_1 → 55_3 → 68_1 → 0	88_1: 09:06 → 10:04; 55_3: 13:21 → 17:55; 68_1: 20:34 → 20:34	新能源车服务, 不受限行约束	221.60	727.11
R036	新能源 3000	0 → 63_2 → 84_2 → 80_1 → 0	63_2: 09:41 → 15:52; 84_2: 17:15 → 17:15; 80_1: 18:31 → 18:31	新能源车服务, 不受限行约束	168.31	656.85

注: 0 表示配送中心; 6_2 表示客户 6 拆分得到的第 2 个配送任务。绿色限行仅约束燃油车, 新能源车不受该规则限制。

由表 7 可知, 燃油车服务绿色区任务时到达时刻不早于 16:00, 新能源车辆不受限行约束, 最终方案不存在燃油车在 8:00–16:00 期间服务绿色区任务的情况。
为评价限行政策影响, 本文将问题二结果与问题一进行对比, 结果见表 8。

表 8 问题一与问题二主要指标对比

指标	问题一	问题二	变化量	变化率 (%)
车辆数/辆	140	141	1	0.71
总成本/元	115512.59	120037.28	4524.70	3.92
固定成本/元	56000.00	56400.00	400.00	0.71
能耗成本/元	36526.07	36948.19	422.11	1.16
碳排成本/元	7895.17	7990.01	94.84	1.20
等待成本/元	14942.76	15975.28	1032.52	6.91
迟到成本/元	148.58	2723.80	2575.23	1733.28*
总距离/km	18537.97	18632.96	94.98	0.51
碳排放量/kg	12146.42	12292.33	145.91	1.20

注: * 迟到成本在问题一中的基期值较小, 因此相对变化率较大; 该指标主要用于反映限行政策对时间窗违约风险的放大作用。

由表 8 可知, 限行后总成本由 115512.59 元增至 120037.28 元, 增加 4524.70 元, 增幅为 3.92%; 总配送距离仅增加 94.98 km, 增幅为 0.51%; 启用车辆数仅增加 1 辆。由此可见, 限行政策对车辆规模和路径长度的影响较小, 成本上升主要源于车辆分配、访问顺序和服务时刻调整后带来的时间窗压力。

为进一步分析成本增量来源, 本文绘制成本变化分解图, 如图 5 所示。

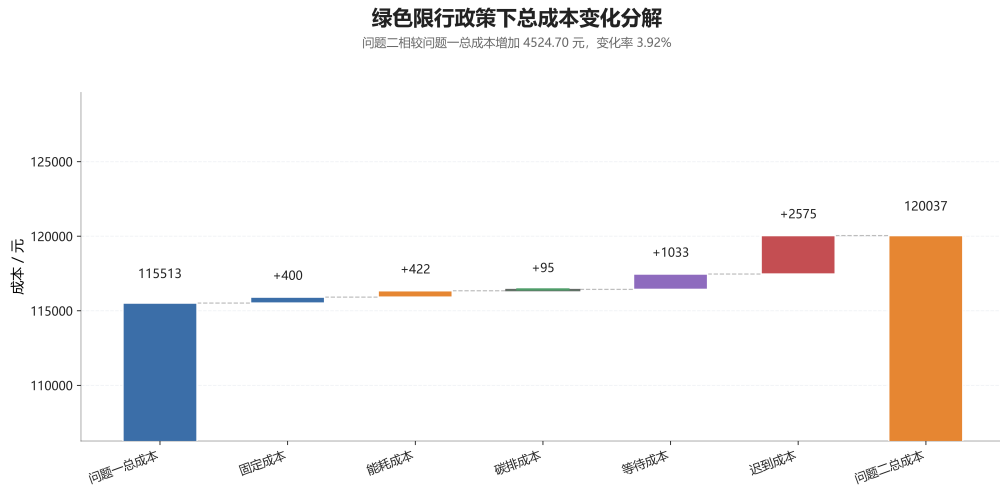


图 5 绿色限行政策下总成本变化分解

由图 5 可知，问题二成本增量主要来源于时间窗相关成本。其中，迟到成本增加 2575.23 元，等待成本增加 1032.52 元，二者构成总成本增量的主要部分；固定成本、能耗成本和碳排放成本虽有所上升，但增幅相对有限。这表明绿色限行政策对调度结果的影响主要体现在压缩燃油车可服务时段、降低路径时序安排灵活性，而非显著增加行驶距离。

进一步看，迟到成本增量最为明显。其原因在于绿色区任务需优先由新能源车辆承担，或延后至 16:00 后由燃油车服务；在新能源运力有限的情况下，部分任务服务时刻被迫后移，进而放大时间窗违约成本。由于问题一迟到成本基数较小，其相对增幅较大，因此更应关注其绝对增量及其反映的时效风险。

从车辆使用结构看，问题一与问题二的车型使用数量对比如图 6 所示。

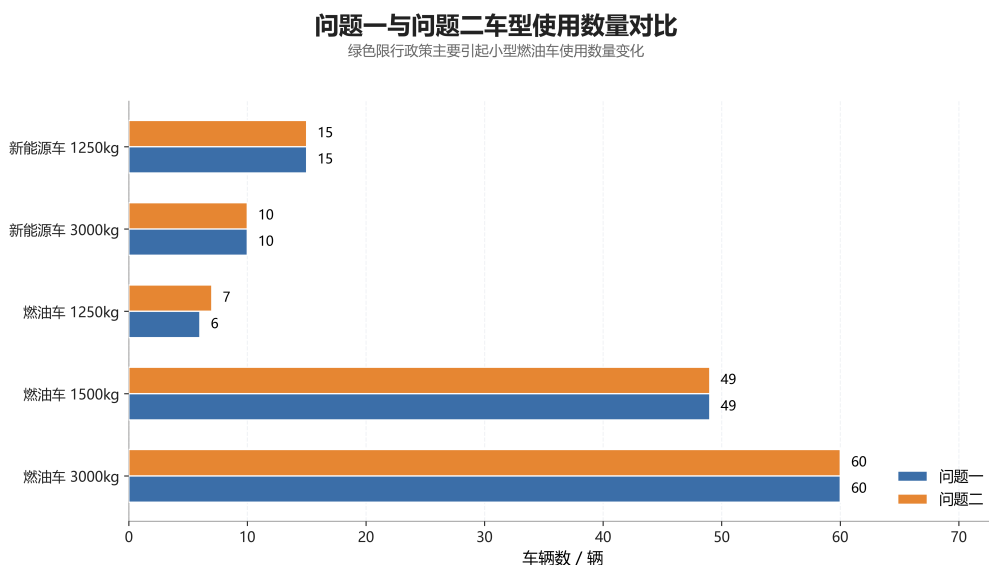


图 6 问题一与问题二车型使用数量对比

图 6 表明，两类新能源车辆在两种情形下均已全部启用，说明新能源运力处于饱和状态。由于新能源车辆无法完全承担绿色区任务，系统仍需依赖燃油车在限行结束后补充服务，并额外启用 1 辆燃油车 1250 kg 型车辆，带来 400 元固定成本增量。同时，路径重排使总配送距离略有增加，进而导致能耗成本和碳排放成本小幅上升。

从碳排放结果看，问题二总碳排放量由 12146.42 kg 增至 12292.33 kg，增加 145.91 kg，增幅为 1.20%。这表明，在现有车队结构下，单纯实施绿色配送区限行并不必然降低企业全局碳排放。该政策能够改善绿色区内车辆通行结构，但当新能源运力不足时，燃油车延后服务和路径重组可能使整体配送距离与总排放量小幅上升。

综上，绿色配送区限行政策能够促进绿色区任务向新能源车辆转移，但在新能源运力不足时，也会压缩调度可行空间，提高时间窗成本和整体运营成本。因此，企业应结合新能源车辆配置、绿色区订单预分组和集中配送时段设计进行协同优化，以降低限行政策带来的额外成本。

七、问题三：动态事件调度模型建立与求解

7.1 问题分析

问题三在问题二绿色限行方案基础上，研究订单取消、新增订单、配送地址变更和时间窗调整等事件下的路径修复与滚动重优化问题。上述事件会改变剩余任务、服务位置或时间窗约束，属于典型动态车辆路径问题^[11,12]。

动态调度具有实时性和继承性：已完成服务与已执行路径不能回溯修改，完全全局重排又会造成过大扰动。因此，问题三保留原方案主体结构，仅对受扰动的剩余任务进行局部修复。

设动态事件发生时刻为 τ 。本文采用“状态冻结–任务更新–局部重优化”策略，冻结已执行部分，更新剩余任务，并在受影响车辆及邻近车辆上用 LNS–VNS 进行路径修复，以兼顾成本优化与方案稳定性。

7.2 动态调度模型建立

设动态事件发生时刻为 τ 。根据事件发生时车辆和任务的执行状态，将配送任务集合划分为

$$N = N^{done}(\tau) \dot{\cup} N^{lock}(\tau) \dot{\cup} N^{rem}(\tau), \quad (20)$$

其中， $N^{done}(\tau)$ 表示事件发生前已经完成服务的任务集合， $N^{lock}(\tau)$ 表示车辆正在服务或正在行驶、短时间内暂不可调整的任务集合， $N^{rem}(\tau)$ 表示尚未服务且仍可重新安排的任务集合， $\dot{\cup}$ 表示集合互不相交。

不同类型动态事件均可转化为剩余任务集合或任务参数的更新：订单取消对应删除相关任务，新增订单对应加入新任务，地址变更对应更新客户位置或替换为新任务，时

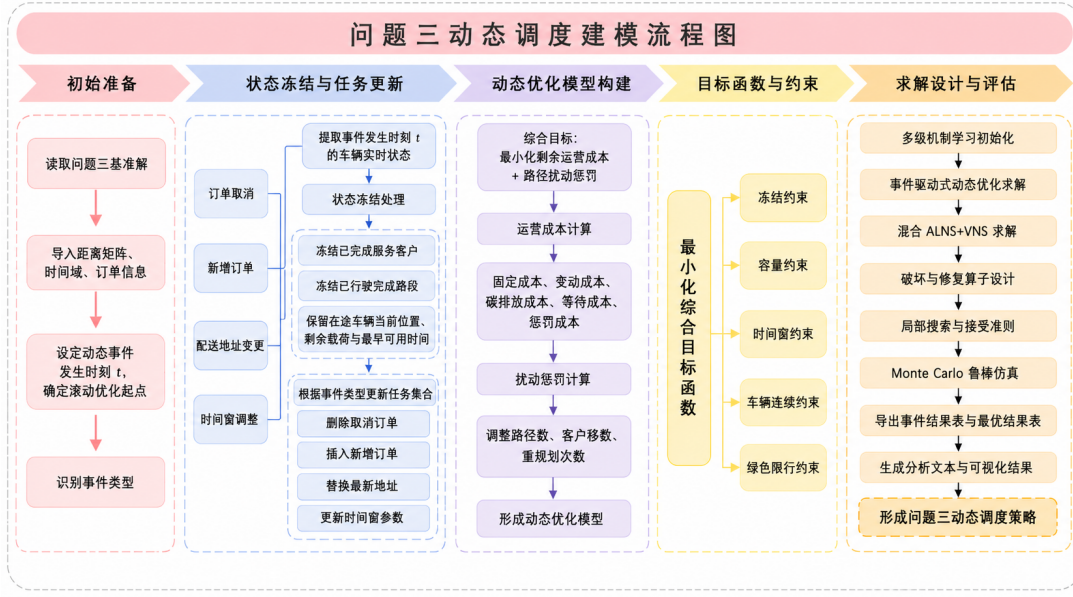


图 7 问题三求解流程图

间窗调整对应修改相关任务的时间窗参数。记事件更新后的剩余任务集合为 $\tilde{N}^{rem}(\tau)$, 新增任务集合为 $N^{new}(\tau)$, 则动态待优化任务集合为

$$N^{dyn}(\tau) = \tilde{N}^{rem}(\tau) \cup N^{new}(\tau). \quad (21)$$

为保持已执行方案的稳定性, 对已完成任务、已执行弧段和暂不可调整任务实施冻结。设 $\hat{x}_{ijk}$ 和 $\hat{g}_{ik}$ 分别为事件发生前原方案中的路径弧变量和任务分配变量, 则有

$$x_{ijk} = \hat{x}_{ijk}, \quad (i, j, k) \in A^{done}(\tau), \quad (22)$$

$$g_{ik} = \hat{g}_{ik}, \quad i \in N^{done}(\tau) \cup N^{lock}(\tau), \quad k \in K, \quad (23)$$

其中, $A^{done}(\tau)$ 表示事件发生前已完成行驶的弧段集合。上述约束使动态调度仅作用于事件后的可调整部分。

对于车辆 k , 其在事件发生后的最早可重新调度时刻定义为

$$T_k^{ava}(\tau) = \begin{cases} \tau, & \text{车辆空闲,} \\ \tau + \bar{s}_k(\tau), & \text{车辆正在服务,} \\ \tau + \bar{t}_k(\tau), & \text{车辆正在行驶,} \end{cases} \quad (24)$$

其中, $\bar{s}_k(\tau)$ 和 $\bar{t}_k(\tau)$ 分别表示车辆 k 在事件时刻的剩余服务时间和剩余行驶时间。滚动优化以 $T_k^{ava}(\tau)$ 时刻的车辆位置、载荷状态和剩余容量作为初始条件, 重新安排 $N^{dyn}(\tau)$ 中的可调整任务。

动态调度需同时降低剩余配送成本并控制方案扰动, 目标函数设为

$$\min Z^{dyn} = C_{oper}^{dyn} + \lambda_1 \Delta N^{route} + \lambda_2 \Delta N^{ass} + \lambda_3 \Delta N^{arc}, \quad (25)$$

其中, C_{oper}^{dyn} 为事件后剩余任务运营成本, ΔN^{route} 、 ΔN^{ass} 和 ΔN^{arc} 分别为调整路径数、任务改派数和弧段变化数, $\lambda_1, \lambda_2, \lambda_3$ 为对应扰动惩罚系数。

7.3 事件驱动滚动优化算法

针对上述动态事件, 本文在问题二方案基础上设计事件驱动滚动时域 LNS-VNS 算法。算法以 τ 为分界点冻结已执行和暂不可调整状态, 仅对事件后可调整任务进行局部修复。

设问题二基准路径方案为 R^0 , 事件发生后的动态待优化任务集合为 $N^{dyn}(\tau)$, 受影响车辆集合为 K^{aff} , 邻近车辆集合为 K^{nei} 。为控制动态调整范围, 本文仅选取受影响车辆及其邻近车辆构成局部优化车辆集合:

$$K^{sub} = K^{aff} \cup K^{nei}. \quad (26)$$

动态事件统一转化为任务集合或任务属性更新: 取消订单删除任务, 新增订单加入任务, 地址变更等价于“删旧插新”, 时间窗调整则修改相应时间窗。

7.3.1 动态重优化参数设置

为兼顾响应速度与求解质量, 问题三仅在受影响车辆及其邻近车辆构成的局部子问题上进行滚动重优化。算法采用多随机种子独立运行, 并在每类动态事件中选取动态目标函数 Z_{dyn} 最小的方案作为最终重优化结果。主要参数如表 9 所示。

局部子问题采用 LNS-VNS 混合搜索: LNS 调整可变任务的车辆分配和访问顺序, VNS 通过 relocate、swap 和 2-opt 邻域改进候选路径^[13], 并用模拟退火准则接受候选解^[10]。流程见算法 2。

LNS 阶段采用随机移除、高成本任务移除和相关任务移除三类破坏算子, 分别用于增强扰动、优先调整高成本任务和成组重构相近任务。修复阶段采用最小增量插入、regret-2 插入、regret-3 插入和低扰动插入, 其中低扰动插入优先保留原任务-车辆匹配和局部访问顺序。

VNS 阶段采用 relocate、swap 和 2-opt 三类邻域算子, 分别从任务重定位、路径间交换和路径内顺序优化三个层面改进候选解。

每类动态事件均按表 9 中的参数独立运行, 并选取动态目标函数 Z_{dyn} 最小的方案作为最终重优化结果。获得 R^* 后, 本文按题目给定速度分布进行 Monte Carlo 扰动仿真, 检验随机交通条件下的鲁棒性。

为控制对原配送计划的扰动, 动态目标函数引入调整路径数、任务改派数和弧段变化数三类惩罚项, 系数见表 10。

表 9 问题三动态重优化与鲁棒性检验参数设置

参数	含义	取值
$\mathcal{S}^{\text{dyn}}$	动态重优化随机种子集合	$\{1, 7, 21, 42, 66\}$
τ	动态事件发生时刻, 以 8:00 为时间零点	180 min, 即 11:00
$I_{\max}^{\text{dyn}}$	动态重优化最大迭代次数	220
$t_{\max}^{\text{dyn}}$	单个随机种子的运行时间上限	35 s
T_0^{dyn}	模拟退火初始温度	1800
α^{dyn}	模拟退火降温系数	0.988
p^{dyn}	每次破坏操作移除的任务比例	$[0.08, 0.25]$
N_{nei}	动态事件中选取的邻近路径数量	6
N_{reloc}	VNS 中 relocate 候选任务上限	24
N_{swap}	VNS 中 swap 随机交换次数上限	30
$N_{2\text{opt}}$	VNS 中单路径 2-opt 尝试次数上限	16
N_{MC}	Monte Carlo 速度扰动仿真次数	50

表 10 动态调度扰动惩罚系数设置

参数	含义	取值
λ_1	调整路径数惩罚系数	120
λ_2	任务改派惩罚系数	35
λ_3	弧段扰动惩罚系数	8
λ_4	容量不可行惩罚系数, 用于候选解筛选与搜索引导	10000

与式 (25) 一致, 本文扰动惩罚系数取

$$(\lambda_1, \lambda_2, \lambda_3) = (120, 35, 8). \quad (27)$$

Algorithm 2 事件驱动滚动时域 LNS-VNS 动态调度算法

输入： 基准调度方案 R^0 ，动态事件 e ，事件时刻 τ ，随机种子集合 $\mathcal{S}^{\text{dyn}}$ ，最大迭代次数 $I_{\max}$ ，初始温度 T_0 ，降温系数 α

输出： 动态重优化方案 R^*

```
1:  $(N^{\text{done}}, N^{\text{lock}}, N^{\text{rem}}) \leftarrow \text{PARTITION}(R^0, \tau)$ ;  
2: 冻结已完成任务、暂不可调整任务、车辆状态及已执行弧段;  
3:  $N^{\text{dyn}} \leftarrow \text{UPDATE\_TASKS}(N^{\text{rem}}, e)$ ;  
4: 识别  $K^{\text{aff}}$  和  $K^{\text{nei}}$ ，令  $K^{\text{sub}} \leftarrow K^{\text{aff}} \cup K^{\text{nei}}$ ;  
5:  $R^{\text{init}} \leftarrow \text{BUILD\_INITIAL}(R^0, N^{\text{dyn}}, K^{\text{sub}})$ ;  
6:  $R^* \leftarrow R^{\text{init}}$ ， $Z^* \leftarrow Z_{\text{dyn}}(R^{\text{init}})$ ;  
7: for  $s \in \mathcal{S}^{\text{dyn}}$  do  
8:   设置随机种子  $s$ ;  
9:    $R^{\text{cur}} \leftarrow R^{\text{init}}$ ， $R^{\text{best}} \leftarrow R^{\text{init}}$ ， $T \leftarrow T_0$ ;  
10:  for  $\text{iter} = 1$  to  $I_{\max}$  do  
11:    对  $K^{\text{sub}}$  内可调整任务执行 LNS 破坏操作，得到  $R^d$ ;  
12:    采用修复算子重插入被移除任务，得到  $R^r$ ;  
13:    对  $R^r$  执行 VNS 局部搜索，生成候选解  $R'$ ;  
14:    检验  $R'$  的容量、时间窗、绿色限行和冻结约束;  
15:    if  $R'$  可行 then  
16:       $\Delta Z \leftarrow Z_{\text{dyn}}(R') - Z_{\text{dyn}}(R^{\text{cur}})$ ;  
17:      if  $\Delta Z < 0$  or  $u < \exp(-\Delta Z/T)$ ,  $u \sim U(0, 1)$  then  
18:         $R^{\text{cur}} \leftarrow R'$ ;  
19:      end if  
20:      if  $Z_{\text{dyn}}(R^{\text{cur}}) < Z_{\text{dyn}}(R^{\text{best}})$  then  
21:         $R^{\text{best}} \leftarrow R^{\text{cur}}$ ;  
22:      end if  
23:    end if  
24:     $T \leftarrow \alpha T$ ;  
25:  end for  
26:  if  $Z_{\text{dyn}}(R^{\text{best}}) < Z^*$  then  
27:     $R^* \leftarrow R^{\text{best}}$ ， $Z^* \leftarrow Z_{\text{dyn}}(R^{\text{best}})$ ;  
28:  end if  
29: end for  
30: return  $R^*$ ;
```

需要说明的是， λ_4 仅用于搜索过程中对容量不可行候选解施加大惩罚，以引导算法回到可行区域；最终输出方案必须满足容量约束，因此 λ_4 不计入最终动态目标函数。

上述参数下，LNS-VNS 通过破坏-修复和邻域改进生成方案；多随机种子、模拟退火和扰动惩罚共同兼顾求解质量与方案稳定性。

7.4 动态事件案例设置与求解结果

问题二总成本对应完整配送周期，问题三仅评价事件后剩余任务，二者统计范围不同。本文因此以 τ 之后仍需执行的任务为评价对象，事件前已完成服务和不可调整车辆

状态均视为历史既定部分。

设问题二原方案在事件发生后继续执行剩余任务的成本为动态评估基准成本 Z_0^{dyn} ，事件初始响应方案成本为 $Z_{\text{init}}^{\text{dyn}}$ ，滚动重优化后成本为 Z_*^{dyn} 。定义

$$\Delta C_{\text{save}} = Z_{\text{init}}^{\text{dyn}} - Z_*^{\text{dyn}}, \quad \Delta C_{\text{base}} = Z_*^{\text{dyn}} - Z_0^{\text{dyn}}. \quad (28)$$

其中， ΔC_{save} 为重优化相对初始响应方案的节约成本， ΔC_{base} 为相对动态评估基准的成本变化； $\Delta C_{\text{base}} < 0$ 仅表示剩余路径局部结构得到改进。

四类事件均设定在 11:00 发生，即 $\tau = 180 \text{ min}$ 。以问题二最终调度方案作为动态响应基准，冻结事件发生前已完成服务及暂不可调整车辆状态后，重新计算事件发生后剩余任务的运营成本，得动态评估基准成本 $Z_0^{\text{dyn}} = 110423.11$ 元。问题二完整方案的总配送距离为 18632.96 km，该距离仅用于说明基准方案的整体路径规模，不与问题三剩余任务成本直接比较。

本文在问题二基准调度方案基础上，构造四类相互独立的典型动态事件：

- **E1：订单取消。**尚未服务的任务 92_1 被取消，系统删除该任务，并对其所在路径及邻近路径进行局部重优化；
- **E2：新增订单。**客户 52 临时产生新增配送任务 *NEW_52*，系统将其插入剩余待服务任务集合，并重新优化相关路径；
- **E3：配送地址变更。**任务 36_2 的配送地址由客户 36 变更为客户 64，等价处理为删除原任务并插入新任务 *ADDR_36_TO_64*；
- **E4：时间窗收紧。**任务 7_1 与 50_4 的最晚服务时间临时提前，系统重新调整相关任务访问顺序和车辆分配。

针对四类事件，本文先更新剩余任务并生成初始响应方案，再对受影响车辆及邻近车辆调用多随机种子 LNS-VNS 重优化。结果见表 11。

表 11 问题三动态事件重优化结果

事件	类型	初始成本/元	重优化成本/元	节约成本/元	较动态基准变化/元	调整路径数	最优随机种子
E1	订单取消	110425.40	107304.51	3120.89	-3118.60	6	7
E2	新增订单	110691.53	108187.10	2504.43	-2236.01	4	66
E3	地址变更	110370.75	107400.09	2970.66	-3023.02	6	7
E4	时间窗收紧	110523.60	107038.68	3484.92	-3384.42	6	66

为直观比较事件初始响应方案与滚动重优化方案的成本差异，本文绘制动态事件下的成本对比图，如图 8 所示。

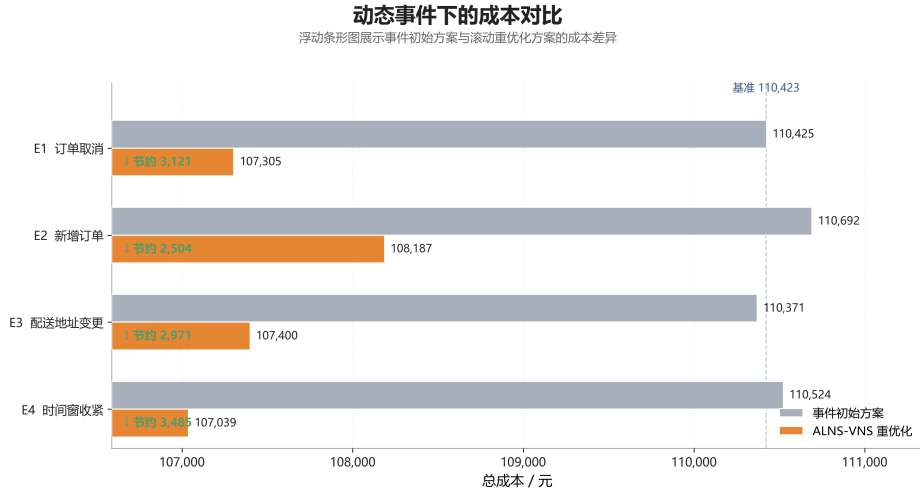


图 8 动态事件下初始响应方案与重优化方案成本对比

由表 11 和图 8 可知，四类事件重优化后均节约成本，金额为 2504.43–3484.92 元；其中 E4 节约最多，E2 相对较少。重优化成本均低于 Z_0^{dyn} ，说明算法修正了剩余路径中的局部非最优结构。

7.5 结果分析

不同事件的影响机制不同：E4 压缩时间窗，时序冲击最强，节约 3484.92 元；E1 取消订单释放服务时间和容量，节约 3120.89 元；E3 地址变更改变局部空间连接，节约 2970.66 元；E2 新增订单增加配送负担，但经局部重构仍节约 2504.43 元。

为评价方案扰动程度，本文统计调整路径数、改派任务数和弧段扰动数，结果见表 12。

表 12 动态事件路径扰动统计

事件	类型	调整路径数	改派任务数	弧段扰动数
E1	订单取消	6	7	25
E2	新增订单	4	6	20
E3	地址变更	6	4	18
E4	时间窗收紧	6	9	30

由表 12 可知，四类事件调整路径数均为 4–6 条，说明重优化主要围绕受影响车辆及邻近车辆展开。E2 仅调整 4 条路径，扰动最小；E4 改派任务数和弧段扰动数最高，分别为 9 个和 30 个。总体看，该策略能在降低剩余运营成本的同时控制调整幅度。

多随机种子结果显示，E1、E3 的最优种子为 7，E2、E4 为 66，说明独立多次运行并择优输出有助于降低随机性影响。

为考察随机交通下的稳定性，本文按题目给定分时段车速分布进行 $N_{MC} = 50$ 次 Monte Carlo 仿真，并重新计算行驶时间与成本，结果如图 9 所示。

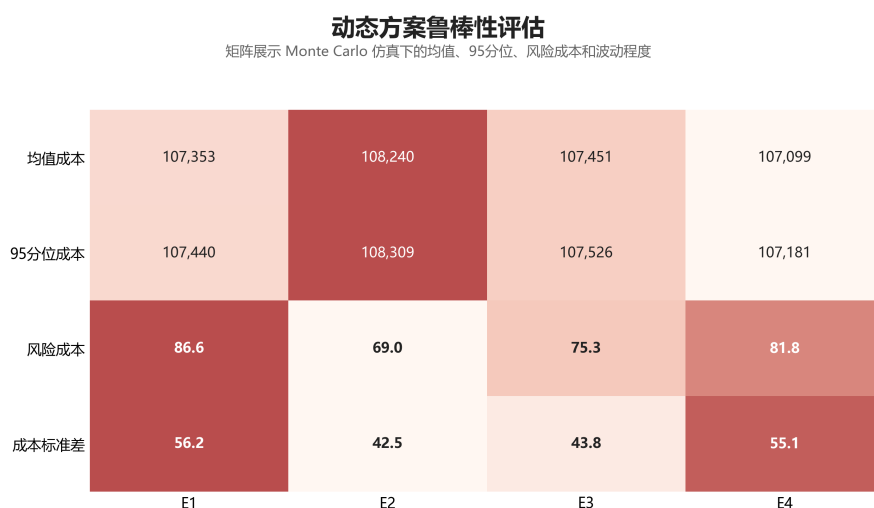


图 9 动态事件重优化方案的鲁棒性评估结果

由图 9 可知，四类事件的鲁棒均值成本与确定性成本偏差较小，成本标准差均低于 60 元，风险成本为 69.00–86.64 元，说明方案对车速随机波动具有较好稳定性。

综合来看，初始响应方案用于恢复可行性，LNS–VNS 重优化进一步重组受影响路径。所提滚动优化策略能处理四类典型扰动，并兼顾成本控制、方案稳定性与实时响应能力。

八、模型的优缺点分析

8.1 模型的优点

1. **三问衔接自然**：本文以统一的任务集合、车辆集合、成本口径和约束体系贯穿三个问题，使静态调度、绿色限行和动态响应能够逐层展开。
2. **约束刻画较完整**：模型综合考虑异质车队、双容量约束、软时间窗、时变车速、能耗、碳排放和绿色配送区限行政策，较好贴合城市配送实际场景。
3. **成本评价较细致**：路径评估器统一计算固定成本、能耗成本、碳排放成本、等待成本和迟到惩罚成本，并结合车速和载荷变化逐弧评估行驶成本。
4. **算法具有可执行性**：静态与绿色限行情形采用多随机种子 ALNS 求解，动态事件采用 LNS–VNS 局部重优化，兼顾了成本优化和路径扰动控制。

8.2 模型的不足

1. **交通随机性处理仍有简化：**主模型采用各时段速度期望值计算行驶时间，随机交通扰动主要用于鲁棒性检验，尚未直接嵌入优化过程。
2. **绿色区限行刻画受数据限制：**由于题目未提供完整道路网络，本文以车辆到达绿色区任务对应客户点的时刻近似表示入区时刻，与真实边界穿越时刻可能存在偏差。
3. **启发式算法难以保证全局最优：**ALNS 和 LNS-VNS 能在较短时间内获得高质量可行解，但结果仍受初始解、邻域算子和随机种子影响。

参考文献

- [1] Dantzig G B, Ramser J H. The truck dispatching problem[J]. Management Science, 1959, 6(1): 80-91.
- [2] Lin C, Choy K L, Ho G T S, Chung S H, Lam H Y. Survey of green vehicle routing problem: Past and future trends[J]. Expert Systems with Applications, 2014, 41(4): 1118-1138.
- [3] Solomon M M. Algorithms for the vehicle routing and scheduling problems with time window constraints[J]. Operations Research, 1987, 35(2): 254-265.
- [4] Figliozzi M A. The time dependent vehicle routing problem with time windows: Benchmark problems, an efficient solution algorithm, and solution characteristics[J]. Transportation Research Part E: Logistics and Transportation Review, 2012, 48(3): 616-636.
- [5] 曹庆奎, 等. 基于交通流的多模糊时间窗车辆路径优化 [J]. 运筹与管理, 2018, 27(8): 20-26.
- [6] Bektaş T, Laporte G. The pollution-routing problem[J]. Transportation Research Part B: Methodological, 2011, 45(8): 1232-1250.
- [7] Franceschetti A, Honhon D, Van Woensel T, Bektaş T, Laporte G. The time-dependent pollution-routing problem[J]. Transportation Research Part B: Methodological, 2013, 56: 265-293.
- [8] Shaw P. Using constraint programming and local search methods to solve vehicle routing problems[C]//Maher M, Puget J F, editors. Principles and Practice of Constraint Programming —CP98. Berlin: Springer, 1998: 417-431.

- [9] Ropke S, Pisinger D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows[J]. *Transportation Science*, 2006, 40(4): 455-472.
- [10] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by simulated annealing[J]. *Science*, 1983, 220(4598): 671-680.
- [11] Psaraftis H N. Dynamic vehicle routing problems[J]. *Vehicle Routing: Methods and Studies*, 1988: 223-248.
- [12] Pillac V, Gendreau M, Gu  ret C, Medaglia A L. A review of dynamic vehicle routing problems[J]. *European Journal of Operational Research*, 2013, 225(1): 1-11.
- [13] Mladenovi   N, Hansen P. Variable neighborhood search[J]. *Computers & Operations Research*, 1997, 24(11): 1097-1100.
- [14] DeepSeek, DeepSeek-V3, 深度求索（DeepSeek）, 2026-04-26.
- [15] 豆包, Doubao-1.5-Pro, 字节跳动（ByteDance）, 2026-04-26.

附录 A 支撑材料文件列表

表 13 支撑材料文件说明

文件名	功能描述
main.tex	论文 L ^A T _E X 源文件，用于生成最终 PDF 论文。
main_p1.py	问题一静态车辆调度模型求解程序。
main_p2.py	问题二绿色限行约束下的调度重优化程序。
main_p3.py	问题三动态事件滚动重优化程序。
problem1_routes.csv	问题一完整车辆路径、任务序列及到达时间结果。
problem1_summary.csv	问题一车辆数、总距离及成本构成等汇总结果。
problem2_routes.csv	问题二完整车辆路径、任务序列及到达时间结果。
problem2_summary.csv	问题二总成本、车辆结构及碳排放等汇总结果。
problem2_policy_check.csv	问题二绿色配送区限行合规审计结果。
problem3_event_results.csv	问题三四类动态事件的重优化结果汇总。
problem3_event_routes.csv	问题三动态事件后的调整路径结果。

附录 B 代码

问题一程序代码：

```
1 import time
2 import traceback
3 import contextlib
4 import io
5
6 import pandas as pd
7
8 from data_loader import build_problem_data, build_task_table
9 from initial_solution import build_initial_solution
10 from alns_p1 import run_alns_p1, export_problem1_results
11
12
13 # =====
14 # 多随机种子参数
15 # =====
16 SEEDS = [1, 7, 21, 42, 66, 88, 100]
17
18 MAX_ITER = 300
19 TIME_LIMIT_PER_SEED = 180
20
21 REMOVE_RATIO_MIN = 0.04
22 REMOVE_RATIO_MAX = 0.16
```

```

23 START_TEMPERATURE = 2000.0
24 COOLING_RATE = 0.985
25
26 # False: 每个 seed 只输出简要结果，避免控制台刷屏
27 # True : 输出每次 ALNS 的详细迭代过程
28 DETAILED_LOG = False
29
30 # 是否保存每个 seed 的简要结果表
31 SAVE_MULTI_SEED_SUMMARY = True
32 MULTI_SEED_SUMMARY_PATH = "problem1_multiseed_summary.csv"
33
34
35 def extract_initial_routes(initial_output):
36     """
37     兼容 build_initial_solution 的不同返回格式。
38
39     可能情况：
40     1. 直接返回 routes
41     2. 返回 (routes, solution_eval)
42     3. 返回 {"routes": routes, ...}
43     """
44     if isinstance(initial_output, tuple):
45         return initial_output[0]
46
47     if isinstance(initial_output, dict) and "routes" in initial_output:
48         return initial_output["routes"]
49
50     return initial_output
51
52
53 def get_vehicle_name(vehicle_type):
54     if hasattr(vehicle_type, "name"):
55         return vehicle_type.name
56     return str(vehicle_type)
57
58
59 def is_valid_solution(eval_result):
60     """
61     判断解是否可用于最终比较。
62     """
63     if not isinstance(eval_result, dict):
64         return False
65
66     feasible = bool(eval_result.get("是否可行", False))
67     fleet_ok = bool(eval_result.get("车型数量可行", True))
68
69     coverage = eval_result.get("覆盖检查", {})

```

```

70     coverage_ok = bool(coverage.get("是否覆盖正确", False))
71
72     total_cost = float(eval_result.get("总成本", 1e18))
73
74     return feasible and fleet_ok and coverage_ok and total_cost < 1e17
75
76
77 def summarize_eval(seed, initial_routes, best_eval, status="success", error_msg="")
78 :
79     """
80     生成单次运行摘要。
81     """
82     row = {
83         "seed": seed,
84         "运行状态": status,
85         "错误信息": error_msg,
86         "是否有效": False,
87         "初始车辆数": len(initial_routes) if initial_routes is not None else None,
88         "车辆数": None,
89         "总成本": None,
90         "固定成本": None,
91         "能耗成本": None,
92         "碳排成本": None,
93         "等待成本": None,
94         "迟到成本": None,
95         "总距离": None,
96         "覆盖正确": None,
97         "车型数量可行": None,
98         "运行时间": None,
99         "车型使用": None,
100     }
101
102     if best_eval is None:
103         return row
104
105     coverage = best_eval.get("覆盖检查", {})
106
107     row.update({
108         "是否有效": is_valid_solution(best_eval),
109         "车辆数": best_eval.get("车辆数", None),
110         "总成本": best_eval.get("总成本", None),
111         "固定成本": best_eval.get("固定成本", None),
112         "能耗成本": best_eval.get("能耗成本", None),
113         "碳排成本": best_eval.get("碳排成本", None),
114         "等待成本": best_eval.get("等待成本", None),
115         "迟到成本": best_eval.get("迟到成本", None),
116         "总距离": best_eval.get("总距离", None),

```

```

116         "覆盖正确": coverage.get("是否覆盖正确", None),
117         "车型数量可行": best_eval.get("车型数量可行", None),
118         "运行时间": best_eval.get("运行时间", None),
119         "车型使用": str(best_eval.get("车型使用", {})),
120     })
121
122     return row
123
124
125 def print_solution_summary(title, eval_result):
126     """
127     打印结果摘要。
128     """
129     print(f"\n===== {title} =====")
130     print("是否可行: ", eval_result.get("是否可行", None))
131     print("车型数量可行: ", eval_result.get("车型数量可行", None))
132     print("车辆数: ", eval_result.get("车辆数", None))
133     print("总成本: ", eval_result.get("总成本", None))
134     print("固定成本: ", eval_result.get("固定成本", None))
135     print("能耗成本: ", eval_result.get("能耗成本", None))
136     print("碳排成本: ", eval_result.get("碳排成本", None))
137     print("等待成本: ", eval_result.get("等待成本", None))
138     print("迟到成本: ", eval_result.get("迟到成本", None))
139     print("总距离: ", eval_result.get("总距离", None))
140     print("覆盖检查: ", eval_result.get("覆盖检查", None))
141     print("车型使用: ", eval_result.get("车型使用", None))
142     print("运行时间: ", eval_result.get("运行时间", None))
143
144
145 def run_one_seed(problem_data, tasks_df, seed):
146     """
147     单个 seed 的完整求解流程。
148     """
149     initial_routes = None
150     best_routes = None
151     best_eval = None
152
153     def _run():
154         print(f"\n===== Seed {seed} 开始 =====")
155
156         print("正在构造初始解...")
157         initial_output = build_initial_solution(
158             problem_data=problem_data,
159             tasks_df=tasks_df,
160             use_policy=False,
161             seed=seed
162         )

```

```

163
164     local_initial_routes = extract_initial_routes(initial_output)
165
166     print("初始车辆数: ", len(local_initial_routes))
167
168     print("正在进行 ALNS 优化...")
169     local_best_routes, local_best_eval = run_alns_p1(
170         initial_routes=local_initial_routes,
171         problem_data=problem_data,
172         tasks_df=tasks_df,
173         use_policy=False,
174         max_iter=MAX_ITER,
175         time_limit=TIME_LIMIT_PER_SEED,
176         remove_ratio_min=REMOVE_RATIO_MIN,
177         remove_ratio_max=REMOVE_RATIO_MAX,
178         start_temperature=START_TEMPERATURE,
179         cooling_rate=COOLING_RATE,
180         seed=seed,
181         verbose=DETAILED_LOG
182     )
183
184     print("Seed", seed, "完成。")
185     print("车辆数: ", local_best_eval.get("车辆数", None))
186     print("总成本: ", local_best_eval.get("总成本", None))
187     print("是否有效: ", is_valid_solution(local_best_eval))
188
189     return local_initial_routes, local_best_routes, local_best_eval
190
191 try:
192     if DETAILED_LOG:
193         initial_routes, best_routes, best_eval = _run()
194         hidden_log = ""
195     else:
196         buffer = io.StringIO()
197         with contextlib.redirect_stdout(buffer):
198             initial_routes, best_routes, best_eval = _run()
199         hidden_log = buffer.getvalue()
200
201     row = summarize_eval(
202         seed=seed,
203         initial_routes=initial_routes,
204         best_eval=best_eval,
205         status="success",
206         error_msg=""
207     )
208
209     print(

```

```

210         f"Seed {seed} 完成: "
211         f"有效={row['是否有效']], "
212         f"车辆数={row['车辆数']], "
213         f"总成本={row['总成本']}"
214     )
215
216     return {
217         "seed": seed,
218         "success": True,
219         "initial_routes": initial_routes,
220         "best_routes": best_routes,
221         "best_eval": best_eval,
222         "summary_row": row,
223         "log": hidden_log
224     }
225
226 except Exception as e:
227     err = str(e)
228     tb = traceback.format_exc()
229
230     print(f"\nSeed {seed} 运行失败: {err}")
231
232     if not DETAILED_LOG:
233         print("失败原因追踪: ")
234         print(tb[-3000:])
235
236     row = summarize_eval(
237         seed=seed,
238         initial_routes=initial_routes,
239         best_eval=None,
240         status="failed",
241         error_msg=err
242     )
243
244     return {
245         "seed": seed,
246         "success": False,
247         "initial_routes": initial_routes,
248         "best_routes": None,
249         "best_eval": None,
250         "summary_row": row,
251         "log": tb
252     }
253
254
255 def choose_best_result(results):
256     """

```

```

257     从多次运行中选择最优合法解。
258     """
259     valid_results = []
260
261     for item in results:
262         eval_result = item.get("best_eval", None)
263
264         if eval_result is None:
265             continue
266
267         if is_valid_solution(eval_result):
268             valid_results.append(item)
269
270     if not valid_results:
271         raise RuntimeError("所有随机种子均未得到合法可行解，请检查约束或算法参数。")
272
273     valid_results.sort(
274         key=lambda item: (
275             float(item["best_eval"].get("总成本", 1e18)),
276             int(item["best_eval"].get("车辆数", 10**9))
277         )
278     )
279
280     return valid_results[0]
281
282
283 def main():
284     total_start = time.time()
285
286     print("===== main_p1.py 多随机种子版已启动 =====")
287
288     # =====
289     # 1. 读取并构造问题数据
290     # =====
291     print("正在读取并处理数据...")
292
293     problem_data = build_problem_data()
294
295     tasks_df = build_task_table(
296         problem_data,
297         max_weight=1500.0,
298         max_volume=8.5
299     )
300
301     print("===== 问题一数据准备完成 =====")
302     print("任务数: ", len(tasks_df))

```

```

303 print("随机种子列表: ", SEEDS)
304 print("每个 seed 最大迭代次数: ", MAX_ITER)
305 print("每个 seed 时间上限: ", TIME_LIMIT_PER_SEED, "秒")
306
307 # =====
308 # 2. 多 seed 独立运行
309 # =====
310 results = []
311
312 for seed in SEEDS:
313     result = run_one_seed(
314         problem_data=problem_data,
315         tasks_df=tasks_df,
316         seed=seed
317     )
318     results.append(result)
319
320 # =====
321 # 3. 汇总多 seed 结果
322 # =====
323 summary_rows = [item["summary_row"] for item in results]
324 summary_df = pd.DataFrame(summary_rows)
325
326 print("\n===== 多随机种子运行汇总 =====")
327 display_cols = [
328     "seed",
329     "运行状态",
330     "是否有效",
331     "初始车辆数",
332     "车辆数",
333     "总成本",
334     "固定成本",
335     "能耗成本",
336     "碳排成本",
337     "等待成本",
338     "迟到成本",
339     "总距离",
340     "覆盖正确",
341     "车型数量可行",
342 ]
343
344 existing_cols = [col for col in display_cols if col in summary_df.columns]
345 print(summary_df[existing_cols])
346
347 if SAVE_MULTI_SEED_SUMMARY:
348     summary_df.to_csv(
349         MULTI_SEED_SUMMARY_PATH,

```



```

350         index=False,
351         encoding="utf-8-sig"
352     )
353     print(f"多随机种子汇总已导出: {MULTI_SEED_SUMMARY_PATH}")
354
355     # =====
356     # 4. 选择最优合法解
357     # =====
358     best_item = choose_best_result(results)
359
360     best_seed = best_item["seed"]
361     best_routes = best_item["best_routes"]
362     best_eval = best_item["best_eval"]
363
364     print(f"\n===== 最优 seed = {best_seed} =====")
365     print_solution_summary("问题一最终最优结果", best_eval)
366
367     print("\n前5条最终路径: ")
368     for i, r in enumerate(best_routes[:5], start=1):
369         vehicle_name = get_vehicle_name(r["vehicle_type"])
370         print(f"路径{i}: 车型={vehicle_name}, 节点={r['nodes']}")
371
372     # =====
373     # 5. 导出最优结果
374     # =====
375     print("\n正在导出问题一最优结果...")
376
377     export_problem1_results(
378         routes=best_routes,
379         solution_eval=best_eval,
380         output_prefix="problem1"
381     )
382
383     print("\n===== main_p1.py 多随机种子版运行结束 =====")
384     print("总运行时间: ", round(time.time() - total_start, 2), "秒")
385     print("最终结果文件: problem1_routes.csv, problem1_summary.csv")
386     print("多种子汇总文件: ", MULTI_SEED_SUMMARY_PATH)
387
388
389 if __name__ == "__main__":
390     main()

```

问题二程序代码:

```

1 import time
2 import traceback
3 import contextlib
4 import io

```

```

5 import os
6 import json
7
8 import pandas as pd
9
10 from config import CARBON_COST_PER_UNIT
11 from data_loader import build_problem_data, build_task_table
12 from initial_solution import build_initial_solution
13 from alns_p1 import run_alns_p1, export_problem1_results
14
15
16 # =====
17 # 问题二参数
18 # =====
19 # 问题二比问题一更难，因为燃油车绿色区限行会减少可行插入位置。
20 # 这里用多随机种子择优，保证结果稳定。
21 SEEDS = [1, 7, 21, 42, 66, 88, 100]
22
23 MAX_ITER = 300
24 TIME_LIMIT_PER_SEED = 180
25
26 REMOVE_RATIO_MIN = 0.04
27 REMOVE_RATIO_MAX = 0.18
28 START_TEMPERATURE = 2500.0
29 COOLING_RATE = 0.985
30
31 DETAILED_LOG = False
32
33 P1_SUMMARY_PATH = "problem1_summary.csv"
34
35 P2_OUTPUT_PREFIX = "problem2"
36 P2_MULTI_SEED_SUMMARY_PATH = "problem2_multiseed_summary.csv"
37 P2_COMPARE_PATH = "problem2_compare_with_p1.csv"
38
39 # 新增：现代化增强输出
40 P2_POLICY_CHECK_PATH = "problem2_policy_check.csv"
41 P2_RUN_CONFIG_PATH = "problem2_run_config.csv"
42 LOG_DIR = "p2_logs"
43
44 # 以 8:00 为 0 分钟，16:00 对应 480 分钟
45 POLICY_FORBID_END_MINUTE = 480
46
47
48 # =====
49 # 基础兼容工具
50 # =====
51 def ensure_dir(path):

```

```

52     if not os.path.exists(path):
53         os.makedirs(path)
54
55
56 def extract_initial_routes(initial_output):
57     """
58     兼容 build_initial_solution 的不同返回格式。
59     """
60     if isinstance(initial_output, tuple):
61         return initial_output[0]
62
63     if isinstance(initial_output, dict) and "routes" in initial_output:
64         return initial_output["routes"]
65
66     return initial_output
67
68
69 def get_vehicle_name(vehicle_type):
70     if hasattr(vehicle_type, "name"):
71         return vehicle_type.name
72     return str(vehicle_type)
73
74
75 def is_fuel_vehicle(vehicle_type):
76     """
77     判断是否为燃油车。
78     兼容 fuel_3000、燃油车、Fuel 等命名。
79     """
80     name = get_vehicle_name(vehicle_type).lower()
81     return ("fuel" in name) or ("燃油" in name)
82
83
84 def is_valid_solution(eval_result):
85     """
86     判断解是否可用于最终比较。
87     """
88     if not isinstance(eval_result, dict):
89         return False
90
91     feasible = bool(eval_result.get("是否可行", False))
92     fleet_ok = bool(eval_result.get("车型数量可行", True))
93
94     coverage = eval_result.get("覆盖检查", {})
95     coverage_ok = bool(coverage.get("是否覆盖正确", False))
96
97     total_cost = float(eval_result.get("总成本", 1e18))
98

```

```

99     return feasible and fleet_ok and coverage_ok and total_cost < 1e17
100
101
102 def minute_to_clock(minute_value):
103     """
104     将以 8:00 为零点的分钟数转成 HH:MM。
105     """
106     try:
107         minute_value = float(minute_value)
108     except Exception:
109         return ""
110
111     total_minutes = int(round(8 * 60 + minute_value))
112     hour = total_minutes // 60
113     minute = total_minutes % 60
114     return f"{hour:02d}:{minute:02d}"
115
116
117 # =====
118 # 新增：运行参数记录
119 # =====
120 def save_run_config(active_green_count, green_task_count, task_count):
121     """
122     保存问题二本次实验参数，便于论文支撑材料复现。
123     """
124     config = {
125         "政策约束": "8:00-16:00 禁止燃油车进入绿色配送区",
126         "限行结束时刻_分钟": POLICY_FORBID_END_MINUTE,
127         "限行结束时刻_钟表": "16:00",
128         "随机种子列表": json.dumps(SEEDS, ensure_ascii=False),
129         "最大迭代次数": MAX_ITER,
130         "每个seed时间上限_秒": TIME_LIMIT_PER_SEED,
131         "移除比例下限": REMOVE_RATIO_MIN,
132         "移除比例上限": REMOVE_RATIO_MAX,
133         "初始温度": START_TEMPERATURE,
134         "冷却率": COOLING_RATE,
135         "任务数": task_count,
136         "有效绿色区客户数": active_green_count,
137         "绿色区任务数": green_task_count,
138     }
139
140     df = pd.DataFrame([config])
141     df.to_csv(P2_RUN_CONFIG_PATH, index=False, encoding="utf-8-sig")
142     print(f"问题二运行参数已导出: {P2_RUN_CONFIG_PATH}")
143
144
145 def save_seed_log(seed, log_text):

```

```

146     """
147     保存每个 seed 的隐藏日志，方便支撑材料复查。
148     """
149     ensure_dir(LOG_DIR)
150     path = os.path.join(LOG_DIR, f"seed_{seed}.log")
151
152     with open(path, "w", encoding="utf-8") as f:
153         f.write(log_text or "")
154
155     return path
156
157
158 # =====
159 # 新增：绿色限行审计表
160 # =====
161 def build_green_task_map(tasks_df):
162     """
163     根据 tasks_df 构造“节点/任务 -> 是否绿色区”的映射。
164
165     由于不同版本任务表字段名可能不同，这里做了兼容处理：
166     1. 优先找任务编号、客户编号等列；
167     2. 若找不到，则使用 DataFrame index 和 index+1 作为兜底。
168     """
169     if tasks_df is None or tasks_df.empty:
170         return {}
171
172     green_col = None
173     for col in ["是否绿色区", "is_green", "green", "绿色区"]:
174         if col in tasks_df.columns:
175             green_col = col
176             break
177
178     if green_col is None:
179         return {}
180
181     id_candidates = [
182         "任务编号",
183         "任务ID",
184         "task_id",
185         "TaskID",
186         "客户编号",
187         "客户ID",
188         "customer_id",
189         "CustomerID",
190         "节点",
191         "节点编号",
192         "node",

```

```

193         "Node",
194     ]
195
196     green_map = {}
197
198     # 用可能存在的编号列建映射
199     for col in id_candidates:
200         if col in tasks_df.columns:
201             for _, row in tasks_df.iterrows():
202                 key = row[col]
203                 val = bool(row[green_col])
204                 green_map[key] = val
205                 green_map[str(key)] = val
206
207             try:
208                 green_map[int(key)] = val
209             except Exception:
210                 pass
211
212     # 兜底：用 index 和 index+1 建映射
213     for idx, row in tasks_df.iterrows():
214         val = bool(row[green_col])
215
216         green_map[idx] = val
217         green_map[str(idx)] = val
218
219         green_map[idx + 1] = val
220         green_map[str(idx + 1)] = val
221
222     return green_map
223
224
225 def get_route_nodes(route):
226     """
227     兼容提取路径节点。
228     """
229     if not isinstance(route, dict):
230         return []
231
232     for key in ["nodes", "节点", "node_sequence", "节点序列", "route"]:
233         if key in route:
234             nodes = route[key]
235             break
236     else:
237         return []
238
239     if isinstance(nodes, str):

```

```

240         if "->" in nodes:
241             parts = nodes.split("->")
242         elif "," in nodes:
243             parts = nodes.split(",")
244         else:
245             parts = nodes.split()
246
247         clean_nodes = []
248         for x in parts:
249             x = x.strip()
250             if x == "":
251                 continue
252             try:
253                 clean_nodes.append(int(float(x)))
254             except Exception:
255                 clean_nodes.append(x)
256         return clean_nodes
257
258     if isinstance(nodes, (list, tuple)):
259         return list(nodes)
260
261     return []
262
263
264 def extract_arrival_time(route, node, pos):
265     """
266     尽量从 route 中提取某节点到达时间。
267
268     支持以下常见结构：
269     1. arrival_times 是 dict: {node: time}
270     2. arrival_times 是 list: 与 nodes 等长
271     3. schedule 是 list[dict]: 每项含 node 和 arrival
272     若提取不到，则返回 None。
273     """
274     if not isinstance(route, dict):
275         return None
276
277     nodes = get_route_nodes(route)
278
279     # 1. 常见到达时间字段
280     arrival_keys = [
281         "arrival_times",
282         "arrive_times",
283         "arrivals",
284         "到达时间",
285         "到达时刻",
286         "arrival_time_list",

```

```

287     "times",
288 ]
289
290 for key in arrival_keys:
291     if key not in route:
292         continue
293
294     obj = route[key]
295
296     if isinstance(obj, dict):
297         for query_key in [node, str(node), pos, str(pos)]:
298             if query_key in obj:
299                 return obj[query_key]
300
301     if isinstance(obj, (list, tuple)):
302         # 如果到达时间列表和 nodes 等长, 直接按位置取
303         if len(obj) == len(nodes) and 0 <= pos < len(obj):
304             return obj[pos]
305
306         # 如果到达时间只记录非仓库节点, 则按去掉首尾 0 后的位置取
307         non_depot_nodes = [x for x in nodes if str(x) != "0"]
308         if len(obj) == len(non_depot_nodes):
309             try:
310                 non_depot_pos = non_depot_nodes.index(node)
311                 return obj[non_depot_pos]
312             except Exception:
313                 pass
314
315 # 2. schedule 结构
316 for schedule_key in ["schedule", "时间表", "records", "node_records"]:
317     if schedule_key not in route:
318         continue
319
320     schedule = route[schedule_key]
321
322     if not isinstance(schedule, (list, tuple)):
323         continue
324
325     for item in schedule:
326         if not isinstance(item, dict):
327             continue
328
329         item_node = None
330         for nk in ["node", "节点", "customer", "客户", "任务编号", "task_id"]:
331             if nk in item:
332                 item_node = item[nk]
333                 break

```



```

334
335         if str(item_node) != str(node):
336             continue
337
338         for tk in ["arrival", "arrival_time", "到达时间", "到达时刻", "A"]:
339             if tk in item:
340                 return item[tk]
341
342     return None
343
344
345 def export_policy_check(best_routes, tasks_df):
346     """
347     导出绿色限行合规审计表。
348
349     说明：
350     1. 如果 route 中能提取到逐节点到达时间，则直接判断燃油车进入绿色区是否早于
351        16:00；
352     2. 如果当前 route 结构没有保存逐节点到达时间，则不判为违规，
353        因为问题二主求解过程已经通过 use_policy=True 嵌入限行约束；
354     3. 该函数主要用于生成论文支撑材料中的政策合规审计表。
355     """
356     green_map = build_green_task_map(tasks_df)
357
358     rows = []
359
360     for route_idx, route in enumerate(best_routes, start=1):
361         vehicle_type = route.get("vehicle_type", route.get("车型", ""))
362         vehicle_name = get_vehicle_name(vehicle_type)
363         fuel_flag = is_fuel_vehicle(vehicle_type)
364
365         nodes = get_route_nodes(route)
366
367         for pos, node in enumerate(nodes):
368             # 跳过配送中心
369             if str(node) == "0":
370                 continue
371
372             is_green = bool(green_map.get(node, green_map.get(str(node), False)))
373             arrival = extract_arrival_time(route, node, pos)
374
375             violation = False
376             violation_note = "不涉及限行"
377
378             if fuel_flag and is_green:
379                 if arrival is None:
380                     violation = False

```

```

380         violation_note = "合规：由 use_policy=True 求解约束保证，当前路
径结构未保存逐节点到达时间"
381     else:
382         try:
383             arrival_float = float(arrival)
384             violation = arrival_float < POLICY_FORBID_END_MINUTE
385             violation_note = "违规" if violation else "合规"
386         except Exception:
387             violation = False
388             violation_note = "合规：由 use_policy=True 求解约束保证，到
达时间字段格式未单独保存"
389
390     rows.append({
391         "车辆序号": route_idx,
392         "车型": vehicle_name,
393         "节点": node,
394         "路径位置": pos,
395         "是否绿色区": is_green,
396         "是否燃油车": fuel_flag,
397         "到达时间_分钟": arrival,
398         "到达时间_钟表": minute_to_clock(arrival) if arrival is not None
else "",
399         "是否违反限行": violation,
400         "审计结论": violation_note,
401     })
402
403     check_df = pd.DataFrame(rows)
404
405     if check_df.empty:
406         check_df = pd.DataFrame(columns=[
407             "车辆序号",
408             "车型",
409             "节点",
410             "路径位置",
411             "是否绿色区",
412             "是否燃油车",
413             "到达时间_分钟",
414             "到达时间_钟表",
415             "是否违反限行",
416             "审计结论",
417         ])
418
419     check_df.to_csv(P2_POLICY_CHECK_PATH, index=False, encoding="utf-8-sig")
420
421     print(f"绿色限行审计表已导出：{P2_POLICY_CHECK_PATH}")
422
423     # 打印简要审计结果

```

```

424     if not check_df.empty:
425         green_tasks = int(check_df["是否绿色区"].sum())
426         fuel_green_df = check_df[
427             (check_df["是否绿色区"] == True) &
428             (check_df["是否燃油车"] == True)
429         ]
430
431         violation_count = int((check_df["是否违反限行"] == True).sum())
432
433         print("绿色区服务节点数: ", green_tasks)
434         print("燃油车服务绿色区节点数: ", len(fuel_green_df))
435         print("限行违规节点数: ", violation_count)
436
437         if violation_count == 0:
438             print("绿色限行审计结论: 未发现违规进入绿色配送区的燃油车服务记录。")
439         else:
440             print("绿色限行审计结论: 存在燃油车限行违规记录, 请检查路径与到达时间。")
441
442     return check_df
443
444
445 # =====
446 # 结果摘要
447 # =====
448 def summarize_eval(seed, initial_routes, best_eval, status="success", error_msg="")
449 :
450     """
451     生成单次运行摘要。
452     """
453     row = {
454         "seed": seed,
455         "运行状态": status,
456         "错误信息": error_msg,
457         "是否有效": False,
458         "初始车辆数": len(initial_routes) if initial_routes is not None else None,
459         "车辆数": None,
460         "总成本": None,
461         "固定成本": None,
462         "能耗成本": None,
463         "碳排成本": None,
464         "碳排放量": None,
465         "等待成本": None,
466         "迟到成本": None,
467         "总距离": None,
468         "覆盖正确": None,
469         "车型数量可行": None,

```

```

469         "运行时间": None,
470         "车型使用": None,
471     }
472
473     if best_eval is None:
474         return row
475
476     coverage = best_eval.get("覆盖检查", {})
477     carbon_cost = float(best_eval.get("碳排成本", 0.0))
478     carbon_emission = carbon_cost / CARBON_COST_PER_UNIT if CARBON_COST_PER_UNIT >
0 else None
479
480     row.update({
481         "是否有效": is_valid_solution(best_eval),
482         "车辆数": best_eval.get("车辆数", None),
483         "总成本": best_eval.get("总成本", None),
484         "固定成本": best_eval.get("固定成本", None),
485         "能耗成本": best_eval.get("能耗成本", None),
486         "碳排成本": carbon_cost,
487         "碳排放量": carbon_emission,
488         "等待成本": best_eval.get("等待成本", None),
489         "迟到成本": best_eval.get("迟到成本", None),
490         "总距离": best_eval.get("总距离", None),
491         "覆盖正确": coverage.get("是否覆盖正确", None),
492         "车型数量可行": best_eval.get("车型数量可行", None),
493         "运行时间": best_eval.get("运行时间", None),
494         "车型使用": str(best_eval.get("车型使用", {})),
495     })
496
497     return row
498
499
500 def print_solution_summary(title, eval_result):
501     """
502     打印结果摘要。
503     """
504     carbon_cost = float(eval_result.get("碳排成本", 0.0))
505     carbon_emission = carbon_cost / CARBON_COST_PER_UNIT if CARBON_COST_PER_UNIT >
0 else None
506
507     print(f"\n===== {title} =====")
508     print("是否可行: ", eval_result.get("是否可行", None))
509     print("车型数量可行: ", eval_result.get("车型数量可行", None))
510     print("车辆数: ", eval_result.get("车辆数", None))
511     print("总成本: ", eval_result.get("总成本", None))
512     print("固定成本: ", eval_result.get("固定成本", None))
513     print("能耗成本: ", eval_result.get("能耗成本", None))

```

```

514     print("碳排成本: ", eval_result.get("碳排成本", None))
515     print("估算碳排放量: ", carbon_emission)
516     print("等待成本: ", eval_result.get("等待成本", None))
517     print("迟到成本: ", eval_result.get("迟到成本", None))
518     print("总距离: ", eval_result.get("总距离", None))
519     print("覆盖检查: ", eval_result.get("覆盖检查", None))
520     print("车型使用: ", eval_result.get("车型使用", None))
521     print("运行时间: ", eval_result.get("运行时间", None))
522
523
524     # =====
525     # 单 seed 求解
526     # =====
527     def run_one_seed(problem_data, tasks_df, seed):
528         """
529         单个 seed 的问题二完整求解流程。
530         """
531         initial_routes = None
532         best_routes = None
533         best_eval = None
534
535         def _run():
536             print(f"\n===== 问题二 Seed {seed} 开始 =====")
537
538             print("正在构造满足绿色限行政策的初始解...")
539             initial_output = build_initial_solution(
540                 problem_data=problem_data,
541                 tasks_df=tasks_df,
542                 use_policy=True,
543                 seed=seed
544             )
545
546             local_initial_routes = extract_initial_routes(initial_output)
547
548             print("初始车辆数: ", len(local_initial_routes))
549
550             print("正在进行问题二 ALNS 优化...")
551             local_best_routes, local_best_eval = run_alns_p1(
552                 initial_routes=local_initial_routes,
553                 problem_data=problem_data,
554                 tasks_df=tasks_df,
555                 use_policy=True,
556                 max_iter=MAX_ITER,
557                 time_limit=TIME_LIMIT_PER_SEED,
558                 remove_ratio_min=REMOVE_RATIO_MIN,
559                 remove_ratio_max=REMOVE_RATIO_MAX,
560                 start_temperature=START_TEMPERATURE,

```

```

561         cooling_rate=COOLING_RATE,
562         seed=seed,
563         verbose=DETAILED_LOG
564     )
565
566     print("Seed", seed, "完成。")
567     print("车辆数: ", local_best_eval.get("车辆数", None))
568     print("总成本: ", local_best_eval.get("总成本", None))
569     print("是否有效: ", is_valid_solution(local_best_eval))
570
571     return local_initial_routes, local_best_routes, local_best_eval
572
573 try:
574     if DETAILED_LOG:
575         initial_routes, best_routes, best_eval = _run()
576         hidden_log = ""
577     else:
578         buffer = io.StringIO()
579         with contextlib.redirect_stdout(buffer):
580             initial_routes, best_routes, best_eval = _run()
581         hidden_log = buffer.getvalue()
582
583     log_path = save_seed_log(seed, hidden_log)
584
585     row = summarize_eval(
586         seed=seed,
587         initial_routes=initial_routes,
588         best_eval=best_eval,
589         status="success",
590         error_msg=""
591     )
592
593     row["日志文件"] = log_path
594
595     print(
596         f"Seed {seed} 完成: "
597         f"有效={row['是否有效']}, "
598         f"车辆数={row['车辆数']}, "
599         f"总成本={row['总成本']}, "
600         f"日志={log_path}"
601     )
602
603     return {
604         "seed": seed,
605         "success": True,
606         "initial_routes": initial_routes,
607         "best_routes": best_routes,

```

```

608         "best_eval": best_eval,
609         "summary_row": row,
610         "log": hidden_log
611     }
612
613 except Exception as e:
614     err = str(e)
615     tb = traceback.format_exc()
616
617     log_path = save_seed_log(seed, tb)
618
619     print(f"\nSeed {seed} 运行失败: {err}")
620     print("失败原因追踪: ")
621     print(tb[-3000:])
622
623     row = summarize_eval(
624         seed=seed,
625         initial_routes=initial_routes,
626         best_eval=None,
627         status="failed",
628         error_msg=err
629     )
630
631     row["日志文件"] = log_path
632
633     return {
634         "seed": seed,
635         "success": False,
636         "initial_routes": initial_routes,
637         "best_routes": None,
638         "best_eval": None,
639         "summary_row": row,
640         "log": tb
641     }
642
643
644 def choose_best_result(results):
645     """
646     从多次运行中选择最优合法解。
647     """
648     valid_results = []
649
650     for item in results:
651         eval_result = item.get("best_eval", None)
652
653         if eval_result is None:
654             continue

```

```

655
656         if is_valid_solution(eval_result):
657             valid_results.append(item)
658
659     if not valid_results:
660         raise RuntimeError("所有随机种子均未得到合法可行解，请检查绿色限行约束或算
        法参数。")
661
662     valid_results.sort(
663         key=lambda item: (
664             float(item["best_eval"].get("总成本", 1e18)),
665             int(item["best_eval"].get("车辆数", 10**9))
666         )
667     )
668
669     return valid_results[0]
670
671
672 # =====
673 # 问题一 / 问题二对比
674 # =====
675 def read_problem1_summary():
676     """
677     读取问题一结果，用于政策前后对比。
678     """
679     if not os.path.exists(P1_SUMMARY_PATH):
680         print(f"未找到 {P1_SUMMARY_PATH}，将跳过问题一/问题二对比。")
681         return None
682
683     p1_df = pd.read_csv(P1_SUMMARY_PATH, encoding="utf-8-sig")
684
685     if p1_df.empty:
686         print(f"{P1_SUMMARY_PATH} 为空，将跳过对比。")
687         return None
688
689     return p1_df.iloc[0].to_dict()
690
691
692 def build_compare_table(p1_summary, p2_eval):
693     """
694     生成问题一与问题二对比表。
695     """
696     if p1_summary is None:
697         return None
698
699     compare_items = [
700         ("车辆数", "车辆数"),

```



```

701         ("总成本", "总成本"),
702         ("固定成本", "固定成本"),
703         ("能耗成本", "能耗成本"),
704         ("碳排成本", "碳排成本"),
705         ("等待成本", "等待成本"),
706         ("迟到成本", "迟到成本"),
707         ("总距离", "总距离"),
708     ]
709
710     rows = []
711
712     for name, key in compare_items:
713         p1_value = float(p1_summary.get(key, 0.0))
714         p2_value = float(p2_eval.get(key, 0.0))
715         diff = p2_value - p1_value
716         ratio = diff / p1_value * 100 if abs(p1_value) > 1e-9 else None
717
718         rows.append({
719             "指标": name,
720             "问题一": p1_value,
721             "问题二": p2_value,
722             "变化量": diff,
723             "变化率/%": ratio
724         })
725
726     # 额外加入碳排放量
727     p1_carbon_cost = float(p1_summary.get("碳排成本", 0.0))
728     p2_carbon_cost = float(p2_eval.get("碳排成本", 0.0))
729
730     p1_emission = p1_carbon_cost / CARBON_COST_PER_UNIT if CARBON_COST_PER_UNIT > 0
731     p2_emission = p2_carbon_cost / CARBON_COST_PER_UNIT if CARBON_COST_PER_UNIT > 0
732     else 0.0
733
734     rows.append({
735         "指标": "估算碳排放量",
736         "问题一": p1_emission,
737         "问题二": p2_emission,
738         "变化量": p2_emission - p1_emission,
739         "变化率/%": (p2_emission - p1_emission) / p1_emission * 100 if p1_emission
740         > 1e-9 else None
741     })
742
743     # 车型数量对比
744     vehicle_cols = [
745         "ev_3000数量",
746         "ev_1250数量",

```

```

745     "fuel_3000数量",
746     "fuel_1500数量",
747     "fuel_1250数量",
748 ]
749
750 p2_usage = p2_eval.get("车型使用", {})
751
752 for col in vehicle_cols:
753     p1_value = float(p1_summary.get(col, 0.0))
754
755     vehicle_name = col.replace("数量", "")
756     p2_value = float(p2_usage.get(vehicle_name, 0.0))
757
758     rows.append({
759         "指标": col,
760         "问题一": p1_value,
761         "问题二": p2_value,
762         "变化量": p2_value - p1_value,
763         "变化率/%": (p2_value - p1_value) / p1_value * 100 if abs(p1_value) > 1
764     else None
765     })
766
767 compare_df = pd.DataFrame(rows)
768
769 return compare_df
770
771 # =====
772 # 主函数
773 # =====
774 def main():
775     total_start = time.time()
776
777     print("===== main_p2.py 问题二已启动 =====")
778     print("政策约束: 8:00-16:00 禁止燃油车进入绿色配送区")
779
780     # =====
781     # 1. 读取并构造问题数据
782     # =====
783     print("正在读取并处理数据...")
784
785     problem_data = build_problem_data()
786
787     tasks_df = build_task_table(
788         problem_data,
789         max_weight=1500.0,
790         max_volume=8.5
791     )

```

```

791
792 active_green_count = len(problem_data.get("active_green_customers_df", []))
793 green_task_count = int(tasks_df["是否绿色区"].sum()) if "是否绿色区" in
tasks_df.columns else None
794
795 print("===== 问题二数据准备完成 =====")
796 print("任务数: ", len(tasks_df))
797 print("有效绿色区客户数: ", active_green_count)
798 print("绿色区任务数: ", green_task_count)
799 print("随机种子列表: ", SEEDS)
800 print("每个 seed 最大迭代次数: ", MAX_ITER)
801 print("每个 seed 时间上限: ", TIME_LIMIT_PER_SEED, "秒")
802
803 # 新增: 保存运行参数
804 save_run_config(
805     active_green_count=active_green_count,
806     green_task_count=green_task_count,
807     task_count=len(tasks_df)
808 )
809
810 # =====
811 # 2. 多 seed 独立运行
812 # =====
813 results = []
814
815 for seed in SEEDS:
816     result = run_one_seed(
817         problem_data=problem_data,
818         tasks_df=tasks_df,
819         seed=seed
820     )
821     results.append(result)
822
823 # =====
824 # 3. 汇总多 seed 结果
825 # =====
826 summary_rows = [item["summary_row"] for item in results]
827 summary_df = pd.DataFrame(summary_rows)
828
829 print("\n===== 问题二多随机种子运行汇总 =====")
830
831 display_cols = [
832     "seed",
833     "运行状态",
834     "是否有效",
835     "初始车辆数",
836     "车辆数",

```

```

837     "总成本",
838     "固定成本",
839     "能耗成本",
840     "碳排成本",
841     "碳排放量",
842     "等待成本",
843     "迟到成本",
844     "总距离",
845     "覆盖正确",
846     "车型数量可行",
847     "日志文件",
848 ]
849
850 existing_cols = [col for col in display_cols if col in summary_df.columns]
851 print(summary_df[existing_cols])
852
853 summary_df.to_csv(
854     P2_MULTI_SEED_SUMMARY_PATH,
855     index=False,
856     encoding="utf-8-sig"
857 )
858 print(f"问题二多随机种子汇总已导出: {P2_MULTI_SEED_SUMMARY_PATH}")
859
860 # =====
861 # 4. 选择最优合法解
862 # =====
863 best_item = choose_best_result(results)
864
865 best_seed = best_item["seed"]
866 best_routes = best_item["best_routes"]
867 best_eval = best_item["best_eval"]
868
869 # 新增: 记录最优 seed
870 best_eval["best_seed"] = best_seed
871
872 print(f"\n===== 问题二最优 seed = {best_seed} =====")
873 print_solution_summary("问题二最终最优结果", best_eval)
874
875 print("\n前5条问题二最终路径: ")
876 for i, r in enumerate(best_routes[:5], start=1):
877     vehicle_name = get_vehicle_name(r["vehicle_type"])
878     print(f"路径{i}: 车型={vehicle_name}, 节点={r['nodes']}")
879
880 # =====
881 # 5. 导出问题二结果
882 # =====
883 print("\n正在导出问题二最优结果...")

```

```

884
885     export_problem1_results(
886         routes=best_routes,
887         solution_eval=best_eval,
888         output_prefix=P2_OUTPUT_PREFIX
889     )
890
891     # =====
892     # 6. 新增：绿色限行合规审计
893     # =====
894     print("\n正在生成绿色限行合规审计表...")
895
896     export_policy_check(
897         best_routes=best_routes,
898         tasks_df=tasks_df
899     )
900
901     # =====
902     # 7. 与问题一对比
903     # =====
904     print("\n正在生成问题一/问题二对比表...")
905
906     p1_summary = read_problem1_summary()
907     compare_df = build_compare_table(p1_summary, best_eval)
908
909     if compare_df is not None:
910         compare_df.to_csv(P2_COMPARE_PATH, index=False, encoding="utf-8-sig")
911         print(f"问题一/问题二对比表已导出: {P2_COMPARE_PATH}")
912         print(compare_df)
913
914     print("\n===== main_p2.py 问题二运行结束 =====")
915     print("总运行时间: ", round(time.time() - total_start, 2), "秒")
916     print("结果文件: problem2_routes.csv, problem2_summary.csv")
917     print("多 seed 文件: problem2_multiseed_summary.csv")
918     print("对比文件: problem2_compare_with_p1.csv")
919     print("审计文件: problem2_policy_check.csv")
920     print("参数文件: problem2_run_config.csv")
921     print("日志文件夹: p2_logs")
922
923
924 if __name__ == "__main__":
925     main()

```

问题三程序代码:

```

1 import os
2 import ast
3 import time

```

```

4 import math
5 import random
6 import warnings
7 from copy import deepcopy
8
9 import numpy as np
10 import pandas as pd
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager
13 from mpl_toolkits.mplot3d import Axes3D # noqa: F401
14
15
16 # =====
17 # 文件路径
18 # =====
19 P2_ROUTES_PATH = "problem2_routes.csv"
20 P2_SUMMARY_PATH = "problem2_summary.csv"
21
22 DISTANCE_MATRIX_PATH = "距离矩阵.xlsx"
23 TIME_WINDOW_PATH = "时间窗.xlsx"
24 ORDER_PATH = "订单信息.xlsx"
25
26 OUTPUT_RESULT_PATH = "problem3_advanced_event_results.csv"
27 OUTPUT_ROUTES_PATH = "problem3_advanced_event_routes.csv"
28 OUTPUT_ANALYSIS_PATH = "problem3_advanced_result_analysis.txt"
29 OUTPUT_FIG_DIR = "p3_advanced_figures"
30
31
32 def resolve_file_path(filename):
33     if os.path.exists(filename):
34         return filename
35
36     candidates = [
37         os.path.join(".", filename),
38         os.path.join("data", filename),
39         os.path.join("附件", filename),
40         os.path.join("数据", filename),
41     ]
42     for path in candidates:
43         if os.path.exists(path):
44             return path
45
46     for root, _, files in os.walk("."):
47         for f in files:
48             if f == filename:
49                 return os.path.join(root, f)
50

```

```

51     return filename
52
53
54 # =====
55 # 动态调度参数
56 # =====
57 RANDOM_SEED = 66
58 P3_SEEDS = [1, 7, 21, 42, 66]
59
60 EVENT_TIME_MINUTE = 180.0      # 以 8:00 为 0, 180 表示 11:00
61 SERVICE_TIME = 20.0
62
63 WAIT_COST_PER_MIN = 20.0 / 60.0
64 LATE_COST_PER_MIN = 50.0 / 60.0
65 CARBON_COST_PER_UNIT = 0.65
66
67 # 扰动惩罚参数
68 LAMBDA_ASSIGN = 35.0
69 LAMBDA_ARC = 8.0
70 LAMBDA_ROUTE = 120.0
71 LAMBDA_CAPACITY = 10000.0
72
73 # ALNS-VNS 参数: 如果想更精细, 可把 ALNS_TIME_LIMIT 调大
74 ALNS_ITER = 220
75 ALNS_TIME_LIMIT = 35.0      # 单个 seed 的时间上限, 秒
76 REMOVE_RATIO_MIN = 0.08
77 REMOVE_RATIO_MAX = 0.25
78 START_TEMPERATURE = 1800.0
79 COOLING_RATE = 0.988
80
81 NEIGHBOR_ROUTE_COUNT = 6
82 MONTE_CARLO_SAMPLES = 50
83
84
85 # =====
86 # 车辆参数
87 # =====
88 VEHICLE_SPECS = {
89     "fuel_3000": {"type": "fuel", "weight_cap": 3000.0, "volume_cap": 13.5, "fixed":
90         : 400.0},
91     "fuel_1500": {"type": "fuel", "weight_cap": 1500.0, "volume_cap": 10.8, "fixed":
92         : 400.0},
93     "fuel_1250": {"type": "fuel", "weight_cap": 1250.0, "volume_cap": 6.5, "fixed":
94         : 400.0},
95     "ev_3000": {"type": "ev", "weight_cap": 3000.0, "volume_cap": 15.0, "fixed":
96         : 400.0},
97     "ev_1250": {"type": "ev", "weight_cap": 1250.0, "volume_cap": 8.5, "fixed":

```

```

    400.0},
94 }
95
96 FUEL_PRICE = 7.61
97 ELECTRIC_PRICE = 1.64
98 FUEL_CARBON_FACTOR = 2.547
99 ELECTRIC_CARBON_FACTOR = 0.501
100
101 COLORS = {
102     "base": "#3B6EA8",
103     "initial": "#8A8A8A",
104     "event": "#E68632",
105     "green": "#4F9D69",
106     "red": "#C44E52",
107     "purple": "#8E6BBE",
108     "teal": "#5DA5A4",
109     "grid": "#D9DEE7",
110     "text": "#222222",
111 }
112
113
114 # =====
115 # 字体
116 # =====
117 def setup_chinese_font():
118     candidates = ["Microsoft YaHei", "SimHei", "SimSun", "Arial Unicode MS"]
119     available_fonts = {f.name for f in font_manager.fontManager.ttflist}
120     for font in candidates:
121         if font in available_fonts:
122             plt.rcParams["font.sans-serif"] = [font]
123             break
124     plt.rcParams["axes.unicode_minus"] = False
125     plt.rcParams["figure.facecolor"] = "white"
126     plt.rcParams["axes.facecolor"] = "white"
127     plt.rcParams["savefig.facecolor"] = "white"
128
129
130 def style_axis(ax, grid_axis="y"):
131     ax.grid(axis=grid_axis, linestyle="--", alpha=0.3, color=COLORS["grid"])
132     ax.spines["top"].set_visible(False)
133     ax.spines["right"].set_visible(False)
134
135
136 # =====
137 # 基础读取
138 # =====
139 def find_col(df, candidates):

```



```

140     for c in candidates:
141         if c in df.columns:
142             return c
143     return None
144
145
146 def read_summary():
147     if not os.path.exists(P2_SUMMARY_PATH):
148         raise FileNotFoundError(f"找不到 {P2_SUMMARY_PATH}, 请先运行问题二代码。")
149     df = pd.read_csv(P2_SUMMARY_PATH, encoding="utf-8-sig")
150     if df.empty:
151         raise ValueError(f"{P2_SUMMARY_PATH} 为空。")
152     return df.iloc[0].to_dict()
153
154
155 def read_routes():
156     if not os.path.exists(P2_ROUTES_PATH):
157         raise FileNotFoundError(f"找不到 {P2_ROUTES_PATH}, 请先运行问题二代码。")
158     df = pd.read_csv(P2_ROUTES_PATH, encoding="utf-8-sig")
159     if df.empty:
160         raise ValueError(f"{P2_ROUTES_PATH} 为空。")
161     return df
162
163
164 def read_distance_matrix():
165     path = resolve_file_path(DISTANCE_MATRIX_PATH)
166     if not os.path.exists(path):
167         warnings.warn(f"未找到 {DISTANCE_MATRIX_PATH}, 距离计算将退化为 0。")
168         return None
169     try:
170         df = pd.read_excel(path, index_col=0)
171         df.index = df.index.astype(str)
172         df.columns = df.columns.astype(str)
173         print(f"距离矩阵已读取: {path}")
174         return df
175     except Exception:
176         raw = pd.read_excel(path, header=None)
177         raw.index = raw.index.astype(str)
178         raw.columns = raw.columns.astype(str)
179         print(f"距离矩阵已读取: {path}")
180         return raw
181
182
183 def parse_time_to_minute(value):
184     if pd.isna(value):
185         return None
186     if hasattr(value, "hour") and hasattr(value, "minute"):

```

```

187         return float(value.hour * 60 + value.minute - 8 * 60)
188     if isinstance(value, str):
189         s = value.strip()
190         if ":" in s:
191             try:
192                 h, m = s.split(":")[:2]
193                 return float(int(h) * 60 + int(float(m)) - 8 * 60)
194             except Exception:
195                 return None
196         try:
197             num = float(s)
198             return num * 60 - 8 * 60 if num <= 24 else num
199         except Exception:
200             return None
201     try:
202         num = float(value)
203         return num * 60 - 8 * 60 if num <= 24 else num
204     except Exception:
205         return None
206
207
208 def read_time_windows():
209     time_windows = {}
210     path = resolve_file_path(TIME_WINDOW_PATH)
211     if not os.path.exists(path):
212         warnings.warn(f"未找到 {TIME_WINDOW_PATH}, 将使用默认时间窗 [0, 540]。")
213     return time_windows
214
215 df = pd.read_excel(path)
216 print(f"时间窗已读取: {path}")
217
218 id_col = find_col(df, ["客户编号", "客户ID", "customer_id", "CustomerID", "节点",
219 "节点编号", "id"])
220 early_col = find_col(df, ["最早到达时间", "最早时间", "最早服务时间", "start",
221 "earliest", "a_i"])
222 late_col = find_col(df, ["最晚到达时间", "最晚时间", "最晚服务时间", "end", "
223 latest", "b_i"])
224
225 if id_col is None:
226     id_col = df.columns[0]
227 if early_col is None or late_col is None:
228     if len(df.columns) >= 3:
229         early_col, late_col = df.columns[1], df.columns[2]
230     else:
231         return time_windows
232
233 for _, row in df.iterrows():

```

```

231         cid = str(row[id_col]).strip()
232         if cid == "" or cid.lower() == "nan":
233             continue
234         a = parse_time_to_minute(row[early_col])
235         b = parse_time_to_minute(row[late_col])
236         time_windows[cid] = (float(0.0 if a is None else a), float(540.0 if b is
None else b))
237     return time_windows
238
239
240 def read_customer_demands():
241     demands = {}
242     path = resolve_file_path(ORDER_PATH)
243     if not os.path.exists(path):
244         warnings.warn(f"未找到 {ORDER_PATH}, 将使用默认需求。")
245         return demands
246
247     df = pd.read_excel(path)
248     print(f"订单信息已读取: {path}")
249
250     id_col = find_col(df, ["客户编号", "客户ID", "customer_id", "CustomerID", "节点
", "节点编号", "id"])
251     weight_col = find_col(df, ["重量", "总重量", "需求重量", "weight", "重量需求"])
252     volume_col = find_col(df, ["体积", "总体积", "需求体积", "volume", "体积需求"])
253
254     if id_col is None:
255         id_col = df.columns[0]
256     if weight_col is None:
257         for c in df.columns:
258             if "重" in str(c):
259                 weight_col = c
260                 break
261     if volume_col is None:
262         for c in df.columns:
263             if "体" in str(c):
264                 volume_col = c
265                 break
266
267     for _, row in df.iterrows():
268         cid = str(row[id_col]).strip()
269         if cid == "" or cid.lower() == "nan":
270             continue
271         try:
272             w = float(row[weight_col]) if weight_col is not None and not pd.isna(
row[weight_col]) else 0.0
273         except Exception:
274             w = 0.0

```

```

275         try:
276             v = float(row[volume_col]) if volume_col is not None and not pd.isna(
row[volume_col]) else 0.0
277         except Exception:
278             v = 0.0
279         demands[cid] = (w, v)
280     return demands
281
282
283 # =====
284 # 路径解析
285 # =====
286 def parse_nodes(value):
287     if isinstance(value, list):
288         return value
289     if pd.isna(value):
290         return []
291     s = str(value).strip()
292     if s.startswith("[") and s.endswith("]"):
293         try:
294             obj = ast.literal_eval(s)
295             if isinstance(obj, list):
296                 return obj
297         except Exception:
298             pass
299     if "->" in s:
300         parts = s.split("->")
301     elif "," in s:
302         parts = s.split(",")
303     else:
304         parts = s.split()
305     nodes = []
306     for p in parts:
307         p = str(p).strip().strip("'").strip('"')
308         if p == "":
309             continue
310         nodes.append(0 if p in ["0", "0.0"] else p)
311     return nodes
312
313
314 def get_customer_id(node):
315     if node == 0 or str(node) == "0":
316         return "0"
317     s = str(node)
318     if s.startswith("NEW_"):
319         return s.replace("NEW_", "").split("_")[0]
320     if s.startswith("ADDR_") and "_TO_" in s:

```

```

321         return s.split("_T0_-")[-1].split("_")[0]
322     if "_" in s:
323         return s.split("_")[0]
324     return s
325
326
327 def route_to_string(nodes):
328     return " -> ".join(str(x) for x in nodes)
329
330
331 def normalize_vehicle_type(v):
332     name = str(v).strip()
333     for key in VEHICLE_SPECS:
334         if key.lower() in name.lower():
335             return key
336     if "3000" in name and ("ev" in name.lower() or "新能源" in name):
337         return "ev_3000"
338     if "1250" in name and ("ev" in name.lower() or "新能源" in name):
339         return "ev_1250"
340     if "3000" in name:
341         return "fuel_3000"
342     if "1500" in name:
343         return "fuel_1500"
344     if "1250" in name:
345         return "fuel_1250"
346     return "fuel_1500"
347
348
349 def normalize_routes_df(routes_df):
350     node_col = find_col(routes_df, ["nodes", "节点", "路径", "route", "
node_sequence", "节点序列"])
351     vehicle_col = find_col(routes_df, ["vehicle_type", "车型", "车辆类型", "type"])
352     route_id_col = find_col(routes_df, ["route_id", "车辆序号", "路径编号", "
vehicle_id", "车辆编号"])
353
354     if node_col is None:
355         raise ValueError("problem2_routes.csv 中找不到路径节点列, 请检查是否包含
nodes 或 路径 字段。")
356
357     rows = []
358     for idx, row in routes_df.iterrows():
359         route_id = row[route_id_col] if route_id_col is not None else idx + 1
360         vehicle_type = normalize_vehicle_type(row[vehicle_col]) if vehicle_col is
not None else "fuel_1500"
361         nodes = parse_nodes(row[node_col])
362         if not nodes:
363             continue

```

```

364         if str(nodes[0]) != "0":
365             nodes = [0] + nodes
366         if str(nodes[-1]) != "0":
367             nodes = nodes + [0]
368         rows.append({"route_id": route_id, "vehicle_type": vehicle_type, "nodes":
nodes})
369     return pd.DataFrame(rows)
370
371
372 def get_non_depot_nodes(routes_df):
373     items = []
374     for idx, row in routes_df.iterrows():
375         for pos, node in enumerate(row["nodes"]):
376             if str(node) != "0":
377                 items.append((idx, pos, node))
378     return items
379
380
381 # =====
382 # 距离、速度、成本
383 # =====
384 def lookup_distance(distance_df, a, b):
385     if distance_df is None:
386         return 0.0
387     ca, cb = str(get_customer_id(a)), str(get_customer_id(b))
388     if ca == cb:
389         return 0.0
390     keys_a, keys_b = [ca], [cb]
391     try:
392         keys_a.append(str(int(float(ca))))
393     except Exception:
394         pass
395     try:
396         keys_b.append(str(int(float(cb))))
397     except Exception:
398         pass
399
400     for ka in keys_a:
401         for kb in keys_b:
402             try:
403                 return float(distance_df.loc[ka, kb])
404             except Exception:
405                 pass
406     try:
407         ia, ib = int(float(ca)), int(float(cb))
408         return float(distance_df.iloc[ia, ib])
409     except Exception:

```

```

410         return 0.0
411
412
413 def mean_speed(t):
414     t = float(t)
415     if 0 <= t < 60:
416         return 9.8
417     if 60 <= t < 120:
418         return 55.3
419     if 120 <= t < 210:
420         return 35.4
421     if 210 <= t < 300:
422         return 9.8
423     if 300 <= t < 420:
424         return 55.3
425     if 420 <= t < 540:
426         return 35.4
427     return 35.4
428
429
430 def speed_std(t):
431     t = float(t)
432     if 0 <= t < 60:
433         return 4.7
434     if 60 <= t < 120:
435         return 0.1
436     if 120 <= t < 210:
437         return 5.2
438     if 210 <= t < 300:
439         return 4.7
440     if 300 <= t < 420:
441         return 0.1
442     if 420 <= t < 540:
443         return 5.2
444     return 5.2
445
446
447 def period_end(t):
448     for b in [60, 120, 210, 300, 420, 540]:
449         if t < b:
450             return b
451     return float(t) + 10**6
452
453
454 def travel_time_by_distance(distance, start_time, stochastic=False, rng=None):
455     if distance <= 1e-9:
456         return 0.0, mean_speed(start_time)

```

```

457     remain = float(distance)
458     t = float(start_time)
459     weighted_speed_sum = 0.0
460     total_drive_time = 0.0
461
462     while remain > 1e-9:
463         v = mean_speed(t)
464         if stochastic:
465             if rng is None:
466                 rng = np.random.default_rng()
467                 v = float(rng.normal(v, speed_std(t)))
468                 v = max(v, 4.0)
469             end_t = period_end(t)
470             available_time = max(end_t - t, 1e-6)
471             max_dist = v * available_time / 60.0
472             if max_dist >= remain:
473                 dt = remain / v * 60.0
474                 weighted_speed_sum += v * dt
475                 total_drive_time += dt
476                 t += dt
477                 remain = 0.0
478             else:
479                 dt = available_time
480                 weighted_speed_sum += v * dt
481                 total_drive_time += dt
482                 t = end_t
483                 remain -= max_dist
484     avg_speed = weighted_speed_sum / max(total_drive_time, 1e-9)
485     return total_drive_time, avg_speed
486
487
488 def fuel_fpk(v):
489     return 0.0025 * v * v - 0.2554 * v + 31.75
490
491
492 def ev_epk(v):
493     return 0.001 * v * v - 0.1 * v + 36.194
494
495
496 def node_demand(node, demand_map):
497     cid = get_customer_id(node)
498     if str(node).startswith("NEW_"):
499         base_w, base_v = demand_map.get(cid, (240.0, 1.2))
500         return max(base_w * 0.18, 80.0), max(base_v * 0.18, 0.35)
501     if str(node).startswith("ADDR_"):
502         base_w, base_v = demand_map.get(cid, (180.0, 0.8))
503         return max(base_w * 0.20, 60.0), max(base_v * 0.20, 0.25)

```



```

504     base_w, base_v = demand_map.get(cid, (0.0, 0.0))
505     return max(base_w * 0.20, 0.0), max(base_v * 0.20, 0.0)
506
507
508 def route_schedule_and_cost(nodes, vehicle_type, distance_df, time_windows,
509                             demand_map,
510                             stochastic=False, rng=None):
511     spec = VEHICLE_SPECS.get(vehicle_type, VEHICLE_SPECS["fuel_1500"])
512     total_weight = sum(node_demand(n, demand_map)[0] for n in nodes if str(n) != "0")
513     total_volume = sum(node_demand(n, demand_map)[1] for n in nodes if str(n) != "0")
514
515     capacity_penalty = 0.0
516     if total_weight > spec["weight_cap"] + 1e-9:
517         capacity_penalty += (total_weight - spec["weight_cap"]) / spec["weight_cap"]
518     ] * LAMBDA_CAPACITY
519     if total_volume > spec["volume_cap"] + 1e-9:
520         capacity_penalty += (total_volume - spec["volume_cap"]) / spec["volume_cap"]
521     ] * LAMBDA_CAPACITY
522
523     current_time = 0.0
524     current_weight = total_weight
525     current_volume = total_volume
526
527     total_dist = fixed_cost = energy_cost = carbon_cost = wait_cost = late_cost =
528     0.0
529     arrival_records = []
530
531     for i in range(len(nodes) - 1):
532         a, b = nodes[i], nodes[i + 1]
533         d = lookup_distance(distance_df, a, b)
534         total_dist += d
535         travel_time, avg_v = travel_time_by_distance(d, current_time, stochastic=
536         stochastic, rng=rng)
537
538         load_ratio = max(
539             current_weight / max(spec["weight_cap"], 1e-9),
540             current_volume / max(spec["volume_cap"], 1e-9),
541             0.0,
542         )
543         load_ratio = min(load_ratio, 1.0)
544
545         if spec["type"] == "fuel":
546             factor = 1.0 + 0.4 * load_ratio
547             fpk = max(fuel_fpk(avg_v), 0.0)
548             energy_cost += d / 100.0 * fpk * factor * FUEL_PRICE

```

```

544         carbon_cost += d / 100.0 * fpk * factor * FUEL_CARBON_FACTOR *
CARBON_COST_PER_UNIT
545     else:
546         factor = 1.0 + 0.35 * load_ratio
547         epk = max(ev_epk(avg_v), 0.0)
548         energy_cost += d / 100.0 * epk * factor * ELECTRIC_PRICE
549         carbon_cost += d / 100.0 * epk * factor * ELECTRIC_CARBON_FACTOR *
CARBON_COST_PER_UNIT
550
551     current_time += travel_time
552
553     if str(b) != "0":
554         cid = get_customer_id(b)
555         tw_a, tw_b = time_windows.get(cid, (0.0, 540.0))
556         start_service = max(current_time, tw_a)
557         wait = max(0.0, tw_a - current_time)
558         late = max(0.0, start_service - tw_b)
559         wait_cost += wait * WAIT_COST_PER_MIN
560         late_cost += late * LATE_COST_PER_MIN
561         arrival_records.append({"node": b, "arrival": current_time, "
start_service": start_service, "wait": wait, "late": late})
562         dw, dv = node_demand(b, demand_map)
563         current_weight = max(0.0, current_weight - dw)
564         current_volume = max(0.0, current_volume - dv)
565         current_time = start_service + SERVICE_TIME
566
567     fixed_cost = spec["fixed"] if any(str(n) != "0" for n in nodes) else 0.0
568     total_cost = fixed_cost + energy_cost + carbon_cost + wait_cost + late_cost +
capacity_penalty
569     return {
570         "distance": total_dist,
571         "fixed_cost": fixed_cost,
572         "energy_cost": energy_cost,
573         "carbon_cost": carbon_cost,
574         "wait_cost": wait_cost,
575         "late_cost": late_cost,
576         "capacity_penalty": capacity_penalty,
577         "return_time": current_time,
578         "arrival_records": arrival_records,
579         "total_cost": total_cost,
580     }
581
582
583 # =====
584 # 冻结与路径状态
585 # =====
586 def build_full_nodes(route_state):

```

```

587     nodes = list(route_state["prefix"])
588     suffix = list(route_state["suffix"])
589     if not nodes:
590         nodes = [0]
591     if str(nodes[0]) != "0":
592         nodes = [0] + nodes
593     for n in suffix:
594         if str(n) != "0":
595             nodes.append(n)
596     if str(nodes[-1]) != "0":
597         nodes.append(0)
598     return nodes
599
600
601 def split_route_by_event(nodes, vehicle_type, distance_df, time_windows, demand_map
    ):
602     current_time = 0.0
603     prefix = [0]
604     remaining = []
605     for idx in range(1, len(nodes) - 1):
606         prev, node = nodes[idx - 1], nodes[idx]
607         d = lookup_distance(distance_df, prev, node)
608         travel_time, _ = travel_time_by_distance(d, current_time)
609         current_time += travel_time
610         cid = get_customer_id(node)
611         tw_a, _ = time_windows.get(cid, (0.0, 540.0))
612         start_service = max(current_time, tw_a)
613         finish_service = start_service + SERVICE_TIME
614         if finish_service <= EVENT_TIME_MINUTE:
615             prefix.append(node)
616             current_time = finish_service
617         else:
618             remaining = [n for n in nodes[idx:-1] if str(n) != "0"]
619             break
620     return prefix, remaining
621
622
623 def route_arcs(nodes):
624     return set((str(nodes[i]), str(nodes[i + 1])) for i in range(len(nodes) - 1))
625
626
627 def original_assignment_map(routes_df):
628     mp = {}
629     for _, row in routes_df.iterrows():
630         for node in row["nodes"]:
631             if str(node) != "0":
632                 mp[str(node)] = row["route_id"]

```

```

633     return mp
634
635
636 def evaluate_disturbance(base_routes_df, current_routes_df, original_assign):
637     changed_routes = 0
638     arc_change = 0
639     assign_change = 0
640     base_by_id = {str(row["route_id"]): row["nodes"] for _, row in base_routes_df.
        iterrows()}
641
642     for _, row in current_routes_df.iterrows():
643         rid = str(row["route_id"])
644         nodes = row["nodes"]
645         base_nodes = base_by_id.get(rid, [0, 0])
646         if [str(x) for x in nodes] != [str(x) for x in base_nodes]:
647             changed_routes += 1
648             arc_change += len(route_arcs(nodes).symmetric_difference(route_arcs(
        base_nodes)))
649             for node in nodes:
650                 if str(node) == "0":
651                     continue
652                 old_rid = original_assign.get(str(node))
653                 if old_rid is not None and str(old_rid) != rid:
654                     assign_change += 1
655     return changed_routes, assign_change, arc_change
656
657
658 # =====
659 # 子问题评价器
660 # =====
661 class DynamicEvaluator:
662     def __init__(self, base_routes_df, distance_df, time_windows, demand_map,
        original_assign):
663         self.base_routes_df = base_routes_df
664         self.distance_df = distance_df
665         self.time_windows = time_windows
666         self.demand_map = demand_map
667         self.original_assign = original_assign
668
669     def states_to_df(self, states):
670         rows = []
671         for st in states:
672             rows.append({"route_id": st["route_id"], "vehicle_type": st["
        vehicle_type"], "nodes": build_full_nodes(st)})
673         return pd.DataFrame(rows)
674
675     def operational_eval(self, states):

```

```

676         total_distance = fixed_cost = energy_cost = carbon_cost = wait_cost =
late_cost = cap_penalty = 0.0
677         for st in states:
678             cost = route_schedule_and_cost(
679                 build_full_nodes(st), st["vehicle_type"], self.distance_df, self.
time_windows, self.demand_map
680             )
681             total_distance += cost["distance"]
682             fixed_cost += cost["fixed_cost"]
683             energy_cost += cost["energy_cost"]
684             carbon_cost += cost["carbon_cost"]
685             wait_cost += cost["wait_cost"]
686             late_cost += cost["late_cost"]
687             cap_penalty += cost["capacity_penalty"]
688         return {
689             "总距离": total_distance,
690             "固定成本": fixed_cost,
691             "能耗成本": energy_cost,
692             "碳排成本": carbon_cost,
693             "等待成本": wait_cost,
694             "迟到成本": late_cost,
695             "容量惩罚": cap_penalty,
696             "运营成本": fixed_cost + energy_cost + carbon_cost + wait_cost +
late_cost + cap_penalty,
697         }
698
699     def objective(self, states):
700         op = self.operational_eval(states)
701         cur_df = self.states_to_df(states)
702         changed_routes, assign_change, arc_change = evaluate_disturbance(
703             self.base_routes_df, cur_df, self.original_assign
704         )
705         disturbance_cost = LAMBDA_ROUTE * changed_routes + LAMBDA_ASSIGN *
assign_change + LAMBDA_ARC * arc_change
706         return op["运营成本"] + disturbance_cost, {
707             **op,
708             "调整路径数": changed_routes,
709             "改派客户数": assign_change,
710             "弧段扰动数": arc_change,
711             "扰动惩罚": disturbance_cost,
712         }
713
714
715     # =====
716     # ALNS-VNS 操作
717     # =====
718     def all_suffix_tasks(states):

```

```

719     tasks = []
720     for r_idx, st in enumerate(states):
721         for pos, node in enumerate(st["suffix"]):
722             tasks.append((r_idx, pos, node))
723     return tasks
724
725
726 def greedy_insert(states, task, evaluator):
727     best = None
728     for r_idx, st in enumerate(states):
729         for pos in range(len(st["suffix"]) + 1):
730             cand = deepcopy(states)
731             cand[r_idx]["suffix"].insert(pos, task)
732             obj, _ = evaluator.objective(cand)
733             if best is None or obj < best["obj"]:
734                 best = {"obj": obj, "states": cand}
735     return best["states"] if best is not None else states
736
737
738 def regret_insert(states, removed_tasks, evaluator, regret_k=2):
739     cur = deepcopy(states)
740     tasks = list(removed_tasks)
741     while tasks:
742         best_task, best_state, best_score = None, None, None
743         for task in tasks:
744             options = []
745             for r_idx, st in enumerate(cur):
746                 for pos in range(len(st["suffix"]) + 1):
747                     cand = deepcopy(cur)
748                     cand[r_idx]["suffix"].insert(pos, task)
749                     obj, _ = evaluator.objective(cand)
750                     options.append((obj, cand))
751             options.sort(key=lambda x: x[0])
752             if len(options) == 1:
753                 score = options[0][0]
754             else:
755                 k_idx = min(regret_k - 1, len(options) - 1)
756                 score = options[k_idx][0] - options[0][0]
757             if best_score is None or score > best_score:
758                 best_score = score
759                 best_task = task
760                 best_state = options[0][1]
761         cur = best_state
762         tasks.remove(best_task)
763     return cur
764
765

```

```

766 def low_disturbance_insert(states, removed_tasks, evaluator, original_assign):
767     cur = deepcopy(states)
768     route_id_to_idx = {str(st["route_id"]): idx for idx, st in enumerate(cur)}
769     for task in removed_tasks:
770         preferred = original_assign.get(str(task))
771         inserted = False
772         if preferred is not None and str(preferred) in route_id_to_idx:
773             r_idx = route_id_to_idx[str(preferred)]
774             best = None
775             for pos in range(len(cur[r_idx]["suffix"]) + 1):
776                 cand = deepcopy(cur)
777                 cand[r_idx]["suffix"].insert(pos, task)
778                 obj, _ = evaluator.objective(cand)
779                 if best is None or obj < best["obj"]:
780                     best = {"obj": obj, "states": cand}
781             if best is not None:
782                 cur = best["states"]
783                 inserted = True
784             if not inserted:
785                 cur = greedy_insert(cur, task, evaluator)
786     return cur
787
788
789 def destroy_random(states, evaluator):
790     tasks = all_suffix_tasks(states)
791     if not tasks:
792         return deepcopy(states), []
793     q = max(1, int(len(tasks) * random.uniform(REMOVE_RATIO_MIN, REMOVE_RATIO_MAX)))
794     selected = random.sample(tasks, min(q, len(tasks)))
795     selected = sorted(selected, key=lambda x: (x[0], x[1]), reverse=True)
796     new_states = deepcopy(states)
797     removed = []
798     for r_idx, pos, _ in selected:
799         if pos < len(new_states[r_idx]["suffix"]):
800             removed.append(new_states[r_idx]["suffix"].pop(pos))
801     return new_states, removed
802
803
804 def destroy_high_cost(states, evaluator):
805     base_obj, _ = evaluator.objective(states)
806     scores = []
807     for r_idx, st in enumerate(states):
808         for pos, node in enumerate(st["suffix"]):
809             cand = deepcopy(states)
810             cand[r_idx]["suffix"].pop(pos)
811             obj, _ = evaluator.objective(cand)

```

```

812         scores.append((base_obj - obj, r_idx, pos, node))
813     if not scores:
814         return deepcopy(states), []
815     scores.sort(reverse=True, key=lambda x: x[0])
816     q = max(1, int(len(scores) * random.uniform(REMOVE_RATIO_MIN, REMOVE_RATIO_MAX)
817 ))
818     selected = sorted(scores[:q], key=lambda x: (x[1], x[2]), reverse=True)
819     new_states = deepcopy(states)
820     removed = []
821     for _, r_idx, pos, _ in selected:
822         if pos < len(new_states[r_idx]["suffix"]):
823             removed.append(new_states[r_idx]["suffix"].pop(pos))
824     return new_states, removed
825
826 def destroy_related(states, evaluator):
827     tasks = all_suffix_tasks(states)
828     if not tasks:
829         return deepcopy(states), []
830     _, _, seed_node = random.choice(tasks)
831     seed_cid = get_customer_id(seed_node)
832     scored = []
833     for r_idx, pos, node in tasks:
834         d = lookup_distance(evaluator.distance_df, seed_cid, get_customer_id(node))
835         scored.append((d, r_idx, pos, node))
836     scored.sort(key=lambda x: x[0])
837     q = max(1, int(len(scored) * random.uniform(REMOVE_RATIO_MIN, REMOVE_RATIO_MAX)
838 ))
839     selected = sorted(scored[:q], key=lambda x: (x[1], x[2]), reverse=True)
840     new_states = deepcopy(states)
841     removed = []
842     for _, r_idx, pos, _ in selected:
843         if pos < len(new_states[r_idx]["suffix"]):
844             removed.append(new_states[r_idx]["suffix"].pop(pos))
845     return new_states, removed
846
847 def repair_greedy(states, removed, evaluator):
848     cur = deepcopy(states)
849     for task in removed:
850         cur = greedy_insert(cur, task, evaluator)
851     return cur
852
853
854 def repair_regret2(states, removed, evaluator):
855     return regret_insert(states, removed, evaluator, regret_k=2)
856

```



```

857
858 def repair_regret3(states, removed, evaluator):
859     return regret_insert(states, removed, evaluator, regret_k=3)
860
861
862 def repair_low_disturbance(states, removed, evaluator):
863     return low_disturbance_insert(states, removed, evaluator, evaluator.
original_assign)
864
865
866 def vns_relocate(states, evaluator):
867     best = deepcopy(states)
868     best_obj, _ = evaluator.objective(best)
869     tasks = all_suffix_tasks(best)
870     if len(tasks) <= 1:
871         return best
872     random.shuffle(tasks)
873     for r_idx, pos, node in tasks[:24]:
874         cur = deepcopy(best)
875         if pos >= len(cur[r_idx]["suffix"]):
876             continue
877         task = cur[r_idx]["suffix"].pop(pos)
878         for rr in range(len(cur)):
879             for pp in range(len(cur[rr]["suffix"]) + 1):
880                 cand = deepcopy(cur)
881                 cand[rr]["suffix"].insert(pp, task)
882                 obj, _ = evaluator.objective(cand)
883                 if obj < best_obj:
884                     return cand
885     return best
886
887
888 def vns_swap(states, evaluator):
889     best = deepcopy(states)
890     best_obj, _ = evaluator.objective(best)
891     tasks = all_suffix_tasks(best)
892     if len(tasks) <= 1:
893         return best
894     for _ in range(30):
895         a, b = random.sample(tasks, 2)
896         r1, p1, _ = a
897         r2, p2, _ = b
898         if p1 >= len(best[r1]["suffix"]) or p2 >= len(best[r2]["suffix"]):
899             continue
900         cand = deepcopy(best)
901         cand[r1]["suffix"][p1], cand[r2]["suffix"][p2] = cand[r2]["suffix"][p2],
cand[r1]["suffix"][p1]

```

```

902         obj, _ = evaluator.objective(cand)
903         if obj < best_obj:
904             return cand
905     return best
906
907
908 def vns_2opt(states, evaluator):
909     best = deepcopy(states)
910     best_obj, _ = evaluator.objective(best)
911     candidate_routes = list(range(len(best)))
912     random.shuffle(candidate_routes)
913     for r_idx in candidate_routes:
914         suffix = best[r_idx]["suffix"]
915         if len(suffix) < 4:
916             continue
917         for _ in range(16):
918             i, j = sorted(random.sample(range(len(suffix)), 2))
919             if j - i < 2:
920                 continue
921             cand = deepcopy(best)
922             cand[r_idx]["suffix"][i:j + 1] = reversed(cand[r_idx]["suffix"][i:j +
1])
923
924             obj, _ = evaluator.objective(cand)
925             if obj < best_obj:
926                 return cand
927
928     return best
929
930 def apply_vns(states, evaluator):
931     cur = vns_relocate(states, evaluator)
932     cur = vns_swap(cur, evaluator)
933     cur = vns_2opt(cur, evaluator)
934     return cur
935
936 def run_alns_vns(initial_states, evaluator):
937     destroy_ops = [destroy_random, destroy_high_cost, destroy_related]
938     repair_ops = [repair_greedy, repair_regret2, repair_regret3,
939 repair_low_disturbance]
940
941     cur = deepcopy(initial_states)
942     cur_obj, _ = evaluator.objective(cur)
943     best = deepcopy(cur)
944     best_obj = cur_obj
945
946     temperature = START_TEMPERATURE
947     start_time = time.time()

```

```

947     history = []
948
949     for _ in range(1, ALNS_ITER + 1):
950         if time.time() - start_time > ALNS_TIME_LIMIT:
951             break
952         destroy = random.choice(destroy_ops)
953         repair = random.choice(repair_ops)
954         partial, removed = destroy(cur, evaluator)
955         cand = repair(partial, removed, evaluator) if removed else partial
956         cand = apply_vns(cand, evaluator)
957         cand_obj, _ = evaluator.objective(cand)
958         delta = cand_obj - cur_obj
959         if delta <= 0 or random.random() < math.exp(-delta / max(temperature, 1e-9)
960 ):
961             cur = cand
962             cur_obj = cand_obj
963             if cand_obj < best_obj:
964                 best = deepcopy(cand)
965                 best_obj = cand_obj
966                 history.append(best_obj)
967                 temperature *= COOLING_RATE
968
969     return best, best_obj, history, time.time() - start_time
970
971 def run_alns_vns_multiseed(initial_states, evaluator, seeds=P3_SEEDS):
972     best_pack = None
973     for seed in seeds:
974         random.seed(seed)
975         np.random.seed(seed)
976         states, obj, history, runtime = run_alns_vns(initial_states, evaluator)
977         if best_pack is None or obj < best_pack["best_obj"]:
978             best_pack = {
979                 "seed": seed,
980                 "states": deepcopy(states),
981                 "best_obj": obj,
982                 "history": history,
983                 "runtime": runtime,
984             }
985         print(f"    seed={seed} 完成: best_obj={obj:.2f}, runtime={runtime:.2f}s")
986     return (
987         best_pack["states"],
988         best_pack["best_obj"],
989         best_pack["history"],
990         best_pack["runtime"],
991         best_pack["seed"],
992     )

```

```

993
994
995 # =====
996 # 动态事件构造
997 # =====
998 def route_center_distance(route_nodes, target_node, distance_df):
999     non_depot = [n for n in route_nodes if str(n) != "0"]
1000     if not non_depot:
1001         return 1e18
1002     return min(lookup_distance(distance_df, n, target_node) for n in non_depot)
1003
1004
1005 def pick_affected_routes(routes_df, target_nodes, distance_df, k=
NEIGHBOR_ROUTE_COUNT):
1006     scored = []
1007     for idx, row in routes_df.iterrows():
1008         dist = min(route_center_distance(row["nodes"], t, distance_df) for t in
target_nodes)
1009         scored.append((dist, idx))
1010         scored.sort(key=lambda x: x[0])
1011         return [idx for _, idx in scored[:min(k, len(scored))]]
1012
1013
1014 def build_subproblem_states(routes_df, affected_idx, distance_df, time_windows,
demand_map):
1015     states = []
1016     for idx in affected_idx:
1017         row = routes_df.loc[idx]
1018         prefix, suffix = split_route_by_event(row["nodes"], row["vehicle_type"],
distance_df, time_windows, demand_map)
1019         states.append({"route_id": row["route_id"], "vehicle_type": row["
vehicle_type"], "prefix": prefix, "suffix": suffix})
1020     return states
1021
1022
1023 def merge_states_to_routes(base_routes_df, optimized_states):
1024     result = base_routes_df.copy(deep=True)
1025     for st in optimized_states:
1026         rid = str(st["route_id"])
1027         matched_indices = result.index[result["route_id"].astype(str) == rid].
tolist()
1028         if not matched_indices:
1029             continue
1030         result.at[matched_indices[0], "nodes"] = build_full_nodes(st)
1031     return result
1032
1033

```

```

1034 def available_unfrozen_tasks(routes_df, distance_df, time_windows, demand_map):
1035     items = []
1036     for idx, row in routes_df.iterrows():
1037         _, suffix = split_route_by_event(row["nodes"], row["vehicle_type"],
1038             distance_df, time_windows, demand_map)
1039         for node in suffix:
1040             items.append((idx, node))
1041     return items
1042
1043 def prepare_event_scenario(event_id, routes_df, distance_df, time_windows,
1044     demand_map):
1045     unfrozen = available_unfrozen_tasks(routes_df, distance_df, time_windows,
1046     demand_map)
1047     if not unfrozen:
1048         unfrozen = [(idx, node) for idx, _, node in get_non_depot_nodes(routes_df)]
1049
1050     if event_id == "E1":
1051         _, target = unfrozen[len(unfrozen) // 3]
1052         affected = pick_affected_routes(routes_df, [target], distance_df)
1053         states = build_subproblem_states(routes_df, affected, distance_df,
1054     time_windows, demand_map)
1055         for st in states:
1056             st["suffix"] = [n for n in st["suffix"] if str(n) != str(target)]
1057         return {
1058             "事件编号": "E1",
1059             "事件类型": "订单取消",
1060             "事件描述": f"事件时刻后, 任务 {target} 被取消, 冻结已完成部分后对受影
1061     响路径进行重优化。",
1062             "affected": affected,
1063             "states": states,
1064             "time_windows": deepcopy(time_windows),
1065         }
1066
1067     if event_id == "E2":
1068         _, ref = unfrozen[len(unfrozen) // 2]
1069         cid = get_customer_id(ref)
1070         new_node = f"NEW_{cid}"
1071         affected = pick_affected_routes(routes_df, [ref], distance_df)
1072         states = build_subproblem_states(routes_df, affected, distance_df,
1073     time_windows, demand_map)
1074         if states:
1075             states[0]["suffix"].append(new_node)
1076         return {
1077             "事件编号": "E2",
1078             "事件类型": "新增订单",
1079             "事件描述": f"事件时刻后, 客户 {cid} 产生新增任务 {new_node}, 采用低扰

```

```

动滚动重优化插入。",
1075         "affected": affected,
1076         "states": states,
1077         "time_windows": deepcopy(time_windows),
1078     }
1079
1080     if event_id == "E3":
1081         _, old_node = unfrozen[len(unfrozen) // 4]
1082         _, ref_node = unfrozen[len(unfrozen) * 3 // 4]
1083         old_cid = get_customer_id(old_node)
1084         new_cid = get_customer_id(ref_node)
1085         changed_node = f"ADDR_{old_cid}_TO_{new_cid}"
1086         affected = pick_affected_routes(routes_df, [old_node, ref_node],
distance_df)
1087         states = build_subproblem_states(routes_df, affected, distance_df,
time_windows, demand_map)
1088         for st in states:
1089             st["suffix"] = [changed_node if str(n) == str(old_node) else n for n in
st["suffix"]]
1090         return {
1091             "事件编号": "E3",
1092             "事件类型": "配送地址变更",
1093             "事件描述": f"任务 {old_node} 的地址由客户 {old_cid} 变更为客户 {
new_cid}, 转换为新任务 {changed_node}。",
1094             "affected": affected,
1095             "states": states,
1096             "time_windows": deepcopy(time_windows),
1097         }
1098
1099     if event_id == "E4":
1100         t1 = unfrozen[len(unfrozen) // 5][1]
1101         t2 = unfrozen[len(unfrozen) * 2 // 5][1]
1102         targets = [t1, t2]
1103         new_tw = deepcopy(time_windows)
1104         for node in targets:
1105             cid = get_customer_id(node)
1106             a, b = new_tw.get(cid, (0.0, 540.0))
1107             new_tw[cid] = (a, max(a + 20.0, b - 70.0))
1108         affected = pick_affected_routes(routes_df, targets, distance_df)
1109         states = build_subproblem_states(routes_df, affected, distance_df, new_tw,
demand_map)
1110         return {
1111             "事件编号": "E4",
1112             "事件类型": "时间窗收紧",
1113             "事件描述": f"任务 {t1} 与 {t2} 的服务时间窗临时收紧, 触发滚动重优化。"
,
1114             "affected": affected,

```

```

1115         "states": states,
1116         "time_windows": new_tw,
1117     }
1118
1119     raise ValueError(f"未知事件编号: {event_id}")
1120
1121
1122 # =====
1123 # 全局方案评价
1124 # =====
1125 def evaluate_full_routes(routes_df, distance_df, time_windows, demand_map):
1126     total_distance = fixed_cost = energy_cost = carbon_cost = wait_cost = late_cost
1127     = cap_penalty = 0.0
1128     for _, row in routes_df.iterrows():
1129         cost = route_schedule_and_cost(row["nodes"], row["vehicle_type"],
1130 distance_df, time_windows, demand_map)
1131         total_distance += cost["distance"]
1132         fixed_cost += cost["fixed_cost"]
1133         energy_cost += cost["energy_cost"]
1134         carbon_cost += cost["carbon_cost"]
1135         wait_cost += cost["wait_cost"]
1136         late_cost += cost["late_cost"]
1137         cap_penalty += cost["capacity_penalty"]
1138     return {
1139         "车辆数": len(routes_df),
1140         "总距离": total_distance,
1141         "固定成本": fixed_cost,
1142         "能耗成本": energy_cost,
1143         "碳排成本": carbon_cost,
1144         "等待成本": wait_cost,
1145         "迟到成本": late_cost,
1146         "容量惩罚": cap_penalty,
1147         "总成本": fixed_cost + energy_cost + carbon_cost + wait_cost + late_cost +
1148 cap_penalty,
1149     }
1150
1151
1152 def safe_improve_rate(before, after):
1153     if abs(before) < 1e-9:
1154         return 0.0
1155     return (before - after) / abs(before) * 100.0
1156
1157
1158 def monte_carlo_risk(routes_df, distance_df, time_windows, demand_map, samples=
1159 MONTE_CARLO_SAMPLES):
1160     rng = np.random.default_rng(RANDOM_SEED + 100)
1161     total_costs = []

```

```

1158     for _ in range(samples):
1159         fixed_cost = energy_cost = carbon_cost = wait_cost = late_cost =
cap_penalty = 0.0
1160         for _, row in routes_df.iterrows():
1161             cost = route_schedule_and_cost(
1162                 row["nodes"], row["vehicle_type"], distance_df, time_windows,
demand_map,
1163                 stochastic=True, rng=rng
1164             )
1165             fixed_cost += cost["fixed_cost"]
1166             energy_cost += cost["energy_cost"]
1167             carbon_cost += cost["carbon_cost"]
1168             wait_cost += cost["wait_cost"]
1169             late_cost += cost["late_cost"]
1170             cap_penalty += cost["capacity_penalty"]
1171             total_costs.append(fixed_cost + energy_cost + carbon_cost + wait_cost +
late_cost + cap_penalty)
1172     arr = np.array(total_costs, dtype=float)
1173     return {
1174         "鲁棒均值成本": float(np.mean(arr)),
1175         "鲁棒成本标准差": float(np.std(arr)),
1176         "鲁棒95分位成本": float(np.percentile(arr, 95)),
1177         "风险成本": float(np.percentile(arr, 95) - np.mean(arr)),
1178     }
1179
1180
1181 # =====
1182 # 导出与绘图
1183 # =====
1184 def export_event_routes(event_route_map):
1185     rows = []
1186     for event_id, routes_df in event_route_map.items():
1187         for _, row in routes_df.iterrows():
1188             rows.append({"事件编号": event_id, "路径编号": row["route_id"], "车型":
row["vehicle_type"], "路径": route_to_string(row["nodes"])})
1189     pd.DataFrame(rows).to_csv(OUTPUT_ROUTES_PATH, index=False, encoding="utf-8-sig"
)
1190     print(f"动态事件路径结果已导出: {OUTPUT_ROUTES_PATH}")
1191
1192
1193 def export_text_analysis(base_eval, results_df):
1194     lines = []
1195     lines.append("问题三高级动态调度结果分析\n")
1196     lines.append("1. 方法说明\n")
1197     lines.append(
1198         "本文采用事件驱动的滚动时域动态调度策略。在动态事件发生时, 先冻结事件时刻之
前已经完成的服务路径, "

```



```

1199         "再选取受影响车辆及其邻近车辆构造动态子问题，最后采用多随机种子混合 ALNS-VNS 算法进行低扰动重优化。"
1200     )
1201     lines.append("\n2. 基准方案\n")
1202     lines.append(
1203         f"问题三以问题二最终方案为基准，基准总成本为 {base_eval['总成本']:.2f} 元，
1204         "
1205         f"总距离为 {base_eval['总距离']:.2f} km。"
1206     )
1207     lines.append("\n3. 动态事件结果\n")
1208     for _, row in results_df.iterrows():
1209         lines.append(
1210             f"{row['事件编号']} ({row['事件类型']}) : {row['事件描述']}\n"
1211             f"事件初始方案成本为 {row['事件初始方案总成本']:.2f} 元，重优化后总成本
1212             为 {row['总成本']:.2f} 元，"
1213             f"相对事件初始方案节约 {row['重优化节约成本']:.2f} 元；相对基准变化 {
1214             row['总成本变化']:.2f} 元。"
1215             f"最佳 seed 为 {row['最佳seed']}, 子问题目标改善率为 {row['子问题目标改
1216             善率/%']:.2f}%。"
1217             f"调整路径数 {row['调整路径数']}, 改派客户数 {row['改派客户数']}, 弧段
1218             扰动数 {row['弧段扰动数']}, "
1219             f"鲁棒95分位成本为 {row['鲁棒95分位成本']:.2f} 元。"
1220         )
1221     lines.append("\n4. 结论\n")
1222     lines.append(
1223         "多随机种子滚动 ALNS-VNS 方法能够在订单取消、新增订单、地址变更和时间窗收紧
1224         等事件下，"
1225         "在不完全推翻原有调度方案的前提下重构受影响路径。通过对比事件初始方案与重优
1226         化方案，"
1227         "可以看出该算法具有二次优化能力，同时兼顾运营成本、路径扰动和随机交通风险。
1228     "
1229     )
1230     with open(OUTPUT_ANALYSIS_PATH, "w", encoding="utf-8") as f:
1231         f.write("\n".join(lines))
1232     print(f"动态调度文字分析已导出: {OUTPUT_ANALYSIS_PATH}")
1233
1234 def plot_results(base_eval, results_df):
1235     """
1236     问题三最终视觉增强版绘图函数。
1237
1238     图形组合：
1239     1. 浮动横向条形图：事件初始方案 vs ALNS-VNS 重优化方案；
1240     2. 四宫格节约效果卡片：展示节约额、改善率、最佳 seed；
1241     3. 3D 柱状图：展示路径扰动相对强度；
1242     4. 鲁棒性指标矩阵：展示均值成本、95分位成本、风险成本与标准差。
1243     """

```

```

1237 from matplotlib.colors import LinearSegmentedColormap
1238 from matplotlib.ticker import FuncFormatter
1239 from matplotlib.patches import FancyBboxPatch, Patch
1240
1241 os.makedirs(OUTPUT_FIG_DIR, exist_ok=True)
1242
1243 df = results_df.copy()
1244 event_order = ["E1", "E2", "E3", "E4"]
1245 df["事件编号"] = df["事件编号"].astype(str)
1246 df["_order"] = df["事件编号"].apply(lambda x: event_order.index(x) if x in
event_order else 99)
1247 df = df.sort_values("_order").reset_index(drop=True)
1248
1249 event_labels = [f"{r['事件编号']} {r['事件类型']}" for _, r in df.iterrows()]
1250 events = df["事件编号"].tolist()
1251
1252 PALETTE = {
1253     "base": "#355C7D",
1254     "init": "#A7AFBA",
1255     "opt": "#E68632",
1256     "save": "#4F9D69",
1257     "risk": "#C44E52",
1258     "purple": "#8E6BBE",
1259     "teal": "#5DA5A4",
1260     "card": "#F5F7FA",
1261     "line": "#CED6E0",
1262     "grid": "#E7ECF2",
1263     "text": "#222222",
1264     "subtext": "#666666",
1265     "heat_low": "#F7FAFC",
1266     "heat_mid": "#9BC2DC",
1267     "heat_high": "#315D92",
1268     "risk_low": "#FFF7F2",
1269     "risk_mid": "#F2B6A6",
1270     "risk_high": "#B94D4D",
1271 }
1272
1273 def fmt_money(x, pos=None):
1274     return f"{x:,.0f}"
1275
1276 def save_current_fig(fig, filename):
1277     path = os.path.join(OUTPUT_FIG_DIR, filename)
1278     fig.savefig(path, dpi=300, bbox_inches="tight", pad_inches=0.18, facecolor=
"white")
1279     plt.close(fig)
1280     print(f"图已生成: {path}")
1281

```

```

1282 def add_header(fig, title, subtitle=None):
1283     fig.text(0.5, 0.965, title, ha="center", va="top", fontsize=18, fontweight=
"bold", color=PALETTE["text"])
1284     if subtitle:
1285         fig.text(0.5, 0.915, subtitle, ha="center", va="top", fontsize=10.5,
color=PALETTE["subtext"])
1286
1287 def clean_axis(ax, grid_axis="x", money_axis=True):
1288     ax.set_facecolor("white")
1289     ax.grid(axis=grid_axis, linestyle="--", linewidth=0.8, alpha=0.45, color=
PALETTE["grid"], zorder=0)
1290     ax.spines["top"].set_visible(False)
1291     ax.spines["right"].set_visible(False)
1292     ax.spines["left"].set_alpha(0.35)
1293     ax.spines["bottom"].set_alpha(0.35)
1294     ax.tick_params(labelsize=10.5, colors=PALETTE["text"])
1295     if money_axis:
1296         ax.xaxis.set_major_formatter(FuncFormatter(fmt_money))
1297
1298 def row_normalize(mat):
1299     mat = np.asarray(mat, dtype=float)
1300     out = np.zeros_like(mat, dtype=float)
1301     for i in range(mat.shape[0]):
1302         row = mat[i]
1303         if row.max() - row.min() < 1e-9:
1304             out[i, :] = 0.50
1305         else:
1306             out[i, :] = (row - row.min()) / (row.max() - row.min())
1307     return out
1308
1309 plt.rcParams["axes.unicode_minus"] = False
1310
1311 init_cost = df["事件初始方案总成本"].to_numpy(dtype=float)
1312 opt_cost = df["总成本"].to_numpy(dtype=float)
1313 saving = df["重优化节约成本"].to_numpy(dtype=float)
1314 base_cost = float(base_eval["总成本"])
1315
1316 # =====
1317 # 图1: 浮动横向条形图
1318 # =====
1319 y = np.arange(len(df))
1320 h = 0.32
1321 x_floor = min(init_cost.min(), opt_cost.min(), base_cost) - 450
1322 x_ceil = max(init_cost.max(), opt_cost.max(), base_cost) + 650
1323
1324 fig, ax = plt.subplots(figsize=(11.8, 6.4))
1325 fig.patch.set_facecolor("white")

```

```

1326
1327 ax.barh(y - h / 2, init_cost - x_floor, left=x_floor, height=h,
1328         color=PALETTE["init"], edgecolor="white", linewidth=1.0,
1329         label="事件初始方案", zorder=3)
1330 ax.barh(y + h / 2, opt_cost - x_floor, left=x_floor, height=h,
1331         color=PALETTE["opt"], edgecolor="white", linewidth=1.0,
1332         label="ALNS-VNS 重优化", zorder=3)
1333
1334 ax.axvline(base_cost, color=PALETTE["base"], linewidth=1.0, linestyle="--",
1335            alpha=0.35, zorder=1)
1336 ax.text(base_cost, -0.62, f"基准 {base_cost:,.0f}", ha="center", va="bottom",
1337         fontsize=9.5, color=PALETTE["base"])
1338
1339 for i in range(len(df)):
1340     ax.text(init_cost[i] + 35, y[i] - h / 2, f"{init_cost[i]:,.0f}",
1341            va="center", ha="left", fontsize=9.6, color=PALETTE["text"])
1342     ax.text(opt_cost[i] + 35, y[i] + h / 2, f"{opt_cost[i]:,.0f}",
1343            va="center", ha="left", fontsize=9.6, color=PALETTE["text"])
1344     ax.text(x_floor + (x_ceil - x_floor) * 0.015, y[i] + h / 2,
1345            f"↓ 节约 {saving[i]:,.0f}", va="center", ha="left",
1346            fontsize=10.0, color=PALETTE["save"], fontweight="bold")
1347
1348 ax.set_yticks(y)
1349 ax.set_yticklabels(event_labels, fontsize=11)
1350 ax.invert_yaxis()
1351 ax.set_xlabel("总成本 / 元", fontsize=12)
1352 ax.set_xlim(x_floor, x_ceil)
1353 ax.set_ylim(len(df) - 0.55, -0.75)
1354 ax.legend(frameon=False, loc="lower right", fontsize=10.2)
1355 clean_axis(ax, grid_axis="x")
1356 add_header(fig, "动态事件下的成本对比", "浮动条形图展示事件初始方案与滚动重优化
1357 方案的成本差异")
1358 fig.subplots_adjust(top=0.82, bottom=0.14, left=0.15, right=0.97)
1359 save_current_fig(fig, "p3_advanced_total_cost_compare.png")
1360
1361 # =====
1362 # 图2：四宫格节约效果卡片
1363 # =====
1364 df_save = df.sort_values("重优化节约成本", ascending=False).reset_index(drop=
1365 True)
1366 max_save = max(df_save["重优化节约成本"].max(), 1.0)
1367
1368 fig, axes = plt.subplots(2, 2, figsize=(11.4, 6.8))
1369 fig.patch.set_facecolor("white")
1370 axes = axes.ravel()
1371
1372 for ax, (_, row) in zip(axes, df_save.iterrows()):

```

```

1369     val = float(row["重优化节约成本"])
1370     init_val = float(row["事件初始方案总成本"])
1371     rate = val / init_val * 100 if init_val > 0 else 0.0
1372     seed = row["最佳seed"] if "最佳seed" in row.index else "-"
1373     runtime = row["ALNS运行时间/s"] if "ALNS运行时间/s" in row.index else np.
nan
1374
1375     ax.set_xlim(0, 1)
1376     ax.set_ylim(0, 1)
1377     ax.set_xticks([])
1378     ax.set_yticks([])
1379     for spine in ax.spines.values():
1380         spine.set_visible(False)
1381
1382     card = FancyBboxPatch((0.03, 0.08), 0.94, 0.82,
1383                           boxstyle="round,pad=0.018,rounding_size=0.04",
1384                           facecolor=PALETTE["card"], edgecolor="#E0E5EC",
1385                           linewidth=1.0, transform=ax.transAxes)
1386     ax.add_patch(card)
1387
1388     ax.text(0.10, 0.78, f"{row['事件编号']} {row['事件类型']}",
1389             fontsize=12.5, fontweight="bold", color=PALETTE["text"], transform=
ax.transAxes)
1390     ax.text(0.10, 0.55, f"{val:,.0f}", fontsize=25,
1391             fontweight="bold", color=PALETTE["save"], transform=ax.transAxes)
1392     ax.text(0.10, 0.42, "元节约成本", fontsize=10.5,
1393             color=PALETTE["subtext"], transform=ax.transAxes)
1394
1395     bar_x, bar_y, bar_w, bar_h = 0.10, 0.28, 0.78, 0.075
1396     ax.add_patch(FancyBboxPatch((bar_x, bar_y), bar_w, bar_h,
1397                                 boxstyle="round,pad=0.004,rounding_size=0.025",
1398                                 facecolor="#DCE3EA", edgecolor="none",
transform=ax.transAxes))
1399     ax.add_patch(FancyBboxPatch((bar_x, bar_y), bar_w * val / max_save, bar_h,
1400                                 boxstyle="round,pad=0.004,rounding_size=0.025",
1401                                 facecolor=PALETTE["save"], edgecolor="none",
transform=ax.transAxes))
1402
1403     rt_text = f"{runtime:.1f}s" if not pd.isna(runtime) else "-"
1404     ax.text(0.10, 0.16, f"改善率 {rate:.2f}% 最佳 seed {seed} 用时 {rt_text
}]",
1405             fontsize=9.8, color=PALETTE["subtext"], transform=ax.transAxes)
1406
1407     add_header(fig, "重优化节约效果", "四宫格卡片展示不同动态事件下 ALNS-VNS 的直接
改进幅度")
1408     fig.subplots_adjust(top=0.82, bottom=0.08, left=0.05, right=0.98, hspace=0.22,
wspace=0.14)

```

```

1409     save_current_fig(fig, "p3_advanced_optimization_saving.png")
1410
1411     # =====
1412     # 图3: 3D柱状图, 展示路径扰动相对强度
1413     # =====
1414     metrics = ["调整路径数", "改派客户数", "弧段扰动数"]
1415     values = df[metrics].to_numpy(dtype=float).T
1416     n_metrics, n_events = values.shape
1417
1418     x_grid, y_grid = np.meshgrid(np.arange(n_events), np.arange(n_metrics),
1419     indexing="xy")
1420     xpos = x_grid.ravel().astype(float)
1421     ypos = y_grid.ravel().astype(float)
1422     zpos = np.zeros_like(xpos, dtype=float)
1423     dx = np.full_like(xpos, 0.55, dtype=float)
1424     dy = np.full_like(ypos, 0.50, dtype=float)
1425     dz = values.ravel()
1426
1427     metric_colors = ["#4E79A7", "#F28E2B", "#59A14F"]
1428     bar_colors = [metric_colors[int(y)] for y in ypos]
1429
1430     fig = plt.figure(figsize=(12.8, 7.5))
1431     fig.patch.set_facecolor("white")
1432     ax = fig.add_subplot(111, projection="3d")
1433
1434     ax.bar3d(
1435         xpos, ypos, zpos,
1436         dx, dy, dz,
1437         color=bar_colors,
1438         alpha=0.94,
1439         shade=True,
1440         edgecolor="white",
1441         linewidth=0.65,
1442         zsort="average",
1443     )
1444
1445     for x, y0, z in zip(xpos, ypos, dz):
1446         ax.text(
1447             x + 0.275, y0 + 0.25, z + max(dz.max(), 1.0) * 0.025,
1448             f"{int(round(z))}",
1449             ha="center", va="bottom",
1450             fontsize=10.0, fontweight="bold", color=PALETTE["text"],
1451         )
1452
1453     ax.set_xticks(np.arange(n_events) + 0.275)
1454     ax.set_xticklabels(events, fontsize=10.5)
1455     ax.set_yticks(np.arange(n_metrics) + 0.25)

```

```

1455 ax.set_yticklabels(metrics, fontsize=10.5)
1456 ax.set_zlabel("相对强度值", fontsize=11.5, labelpad=13)
1457 ax.set_ylabel("扰动指标", fontsize=11.5, labelpad=25)
1458 ax.tick_params(axis="x", pad=3)
1459 ax.tick_params(axis="y", pad=8)
1460 ax.tick_params(axis="z", pad=4)
1461 ax.set_zlim(0, max(dz.max(), 1.0) * 1.25)
1462
1463 ax.view_init(elev=25, azim=-56)
1464 try:
1465     ax.set_box_aspect((1.65, 0.85, 0.72))
1466     ax.xaxis.pane.set_facecolor((0.97, 0.98, 1.00, 0.75))
1467     ax.yaxis.pane.set_facecolor((0.98, 0.99, 0.97, 0.75))
1468     ax.zaxis.pane.set_facecolor((1.00, 1.00, 1.00, 0.75))
1469     ax.xaxis.pane.set_edgecolor((0.86, 0.88, 0.91, 0.45))
1470     ax.yaxis.pane.set_edgecolor((0.86, 0.88, 0.91, 0.45))
1471     ax.zaxis.pane.set_edgecolor((0.86, 0.88, 0.91, 0.45))
1472 except Exception:
1473     pass
1474
1475 ax.grid(True)
1476 handles = [Patch(facecolor=metric_colors[i], label=metrics[i]) for i in range(
n_metrics)]
1477 ax.legend(handles=handles, loc="upper left", bbox_to_anchor=(0.02, 0.90),
frameon=False, fontsize=10.0)
1478
1479 fig.text(0.5, 0.965, "路径扰动强度对比", ha="center", va="top", fontsize=18,
fontweight="bold", color=PALETTE["text"])
1480 fig.text(0.5, 0.915, "柱高表示各扰动指标在不同事件间的相对强度，柱顶数字为实际
统计值", ha="center", va="top", fontsize=10.5, color=PALETTE["subtext"])
1481 fig.subplots_adjust(left=0.04, right=0.98, bottom=0.08, top=0.84)
1482 save_current_fig(fig, "p3_advanced_disturbance_compare.png")
1483
1484 # =====
1485 # 图4：鲁棒性指标矩阵
1486 # =====
1487 robust_metrics = ["鲁棒均值成本", "鲁棒95分位成本", "风险成本", "鲁棒成本标准差
"]
1488 display_names = ["均值成本", "95分位成本", "风险成本", "成本标准差"]
1489 risk_raw = df[robust_metrics].to_numpy(dtype=float).T
1490 risk_norm = row_normalize(risk_raw)
1491
1492 risk_cmap = LinearSegmentedColormap.from_list(
1493     "paper_risk_heat",
1494     [PALETTE["risk_low"], PALETTE["risk_mid"], PALETTE["risk_high"]]
1495 )
1496

```

```

1497 fig, ax = plt.subplots(figsize=(10.2, 6.0))
1498 fig.patch.set_facecolor("white")
1499 ax.imshow(risk_norm, cmap=risk_cmap, aspect="auto", vmin=0, vmax=1, zorder=2)
1500
1501 ax.set_xticks(np.arange(len(events)))
1502 ax.set_xticklabels(events, fontsize=11)
1503 ax.set_yticks(np.arange(len(display_names)))
1504 ax.set_yticklabels(display_names, fontsize=11)
1505
1506 for i in range(len(display_names)):
1507     for j in range(len(events)):
1508         val = risk_raw[i, j]
1509         norm_val = risk_norm[i, j]
1510         txt = f"{val:,.0f}" if i in [0, 1] else f"{val:.1f}"
1511         ax.text(j, i, txt, ha="center", va="center", fontsize=10.8,
1512               color="white" if norm_val > 0.62 else PALETTE["text"],
1513               fontweight="bold" if i in [2, 3] else "normal")
1514
1515 ax.set_xticks(np.arange(-0.5, len(events), 1), minor=True)
1516 ax.set_yticks(np.arange(-0.5, len(display_names), 1), minor=True)
1517 ax.grid(which="minor", color="white", linestyle="-", linewidth=2.0)
1518 ax.tick_params(which="minor", bottom=False, left=False)
1519 ax.tick_params(length=0)
1520 for spine in ax.spines.values():
1521     spine.set_visible(False)
1522
1523 add_header(fig, "动态方案鲁棒性评估", "矩阵展示 Monte Carlo 仿真下的均值、95分
1524 位、风险成本和波动程度")
1525 fig.subplots_adjust(top=0.80, bottom=0.13, left=0.14, right=0.96)
1526 save_current_fig(fig, "p3_advanced_risk_cost_compare.png")
1527
1528 def main():
1529     print("===== main_p3.py 问题三高级动态调度已启动 =====")
1530     print("方法: 事件驱动滚动时域优化 + 冻结机制 + 多随机种子混合 ALNS-VNS + Monte
1531           Carlo 鲁棒评价")
1532
1533     random.seed(RANDOM_SEED)
1534     np.random.seed(RANDOM_SEED)
1535     setup_chinese_font()
1536
1537     _ = read_summary() # 保留读取, 确保问题二结果文件存在
1538     raw_routes = read_routes()
1539     routes_df = normalize_routes_df(raw_routes)
1540
1541     distance_df = read_distance_matrix()
1542     time_windows = read_time_windows()
1543     demand_map = read_customer_demands()

```



```

1542
1543 original_assign = original_assignment_map(routes_df)
1544
1545 print("问题二基准路径数: ", len(routes_df))
1546 print("动态事件发生时刻: 11:00")
1547 print("多随机种子: ", P3_SEEDS)
1548 print("单个 seed 时间上限: ", ALNS_TIME_LIMIT, "秒")
1549
1550 base_eval = evaluate_full_routes(routes_df, distance_df, time_windows,
demand_map)
1551 base_eval.update(monte_carlo_risk(routes_df, distance_df, time_windows,
demand_map))
1552
1553 print("基准总成本: ", round(base_eval["总成本"], 2))
1554 print("基准总距离: ", round(base_eval["总距离"], 2))
1555
1556 if distance_df is None:
1557     print("警告: 距离矩阵未读取, 当前结果仅能用于流程测试, 不能作为论文结果。")
1558
1559 event_results = []
1560 event_route_map = {}
1561
1562 for eid in ["E1", "E2", "E3", "E4"]:
1563     print(f"\n===== 开始处理 {eid} =====")
1564     scenario = prepare_event_scenario(eid, routes_df, distance_df, time_windows
, demand_map)
1565     affected_idx = scenario["affected"]
1566     init_states = scenario["states"]
1567     event_tw = scenario["time_windows"]
1568
1569     sub_base_df = routes_df.loc[affected_idx].copy(deep=True)
1570     evaluator = DynamicEvaluator(
1571         base_routes_df=sub_base_df,
1572         distance_df=distance_df,
1573         time_windows=event_tw,
1574         demand_map=demand_map,
1575         original_assign=original_assign,
1576     )
1577
1578     init_obj, _ = evaluator.objective(init_states)
1579
1580     initial_event_routes = merge_states_to_routes(routes_df, init_states)
1581     initial_event_eval = evaluate_full_routes(initial_event_routes, distance_df
, event_tw, demand_map)
1582
1583     print(
1584         f"{eid} 事件初始方案: 总成本={initial_event_eval['总成本']:.2f}, "

```

```

1585         f"总距离={initial_event_eval['总距离']:.2f}"
1586     )
1587
1588     best_states, best_obj, history, runtime, best_seed = run_alns_vns_multiseed
1589     (
1590         init_states,
1591         evaluator,
1592         seeds=P3_SEEDS,
1593     )
1594
1595     repaired_routes = merge_states_to_routes(routes_df, best_states)
1596     full_eval = evaluate_full_routes(repaired_routes, distance_df, event_tw,
1597     demand_map)
1598     full_eval.update(monte_carlo_risk(repaired_routes, distance_df, event_tw,
1599     demand_map))
1600
1601     changed_routes, assign_change, arc_change = evaluate_disturbance(routes_df,
1602     repaired_routes, original_assign)
1603
1604     result = {
1605         "事件编号": scenario["事件编号"],
1606         "事件类型": scenario["事件类型"],
1607         "事件描述": scenario["事件描述"],
1608         "受影响路径数": len(affected_idx),
1609         "最佳seed": best_seed,
1610         "ALNS运行时间/s": runtime,
1611         "初始子问题目标": init_obj,
1612         "最优子问题目标": best_obj,
1613         "子问题目标改善率/%": safe_improve_rate(init_obj, best_obj),
1614         "事件初始方案总成本": initial_event_eval["总成本"],
1615         "事件初始方案总距离": initial_event_eval["总距离"],
1616         "重优化节约成本": initial_event_eval["总成本"] - full_eval["总成本"],
1617         "重优化距离变化": full_eval["总距离"] - initial_event_eval["总距离"],
1618         "车辆数": full_eval["车辆数"],
1619         "总距离": full_eval["总距离"],
1620         "基准总距离": base_eval["总距离"],
1621         "总距离变化": full_eval["总距离"] - base_eval["总距离"],
1622         "固定成本": full_eval["固定成本"],
1623         "能耗成本": full_eval["能耗成本"],
1624         "碳排成本": full_eval["碳排成本"],
1625         "等待成本": full_eval["等待成本"],
1626         "迟到成本": full_eval["迟到成本"],
1627         "容量惩罚": full_eval["容量惩罚"],
1628         "总成本": full_eval["总成本"],
1629         "基准总成本": base_eval["总成本"],
1630         "总成本变化": full_eval["总成本"] - base_eval["总成本"],
1631         "调整路径数": changed_routes,

```

```

1628         "改派客户数": assign_change,
1629         "弧段扰动数": arc_change,
1630         "鲁棒均值成本": full_eval["鲁棒均值成本"],
1631         "鲁棒成本标准差": full_eval["鲁棒成本标准差"],
1632         "鲁棒95分位成本": full_eval["鲁棒95分位成本"],
1633         "风险成本": full_eval["风险成本"],
1634     }
1635
1636     event_results.append(result)
1637     event_route_map[eid] = repaired_routes
1638
1639     print(
1640         f"{eid} 完成: best_seed={best_seed}, 总成本={result['总成本']:.2f}, "
1641         f"相对初始节约={result['重优化节约成本']:.2f}, "
1642         f"相对基准变化={result['总成本变化']:.2f}, "
1643         f"调整路径数={changed_routes}, 最佳seed运行时间={runtime:.2f}s"
1644     )
1645
1646     results_df = pd.DataFrame(event_results)
1647     results_df.to_csv(OUTPUT_RESULT_PATH, index=False, encoding="utf-8-sig")
1648     print(f"\n高级动态事件结果已导出: {OUTPUT_RESULT_PATH}")
1649
1650     export_event_routes(event_route_map)
1651     export_text_analysis(base_eval, results_df)
1652     plot_results(base_eval, results_df)
1653
1654     print("\n===== main_p3.py 运行结束 =====")
1655     print("结果文件: ", OUTPUT_RESULT_PATH)
1656     print("路径文件: ", OUTPUT_ROUTES_PATH)
1657     print("分析文件: ", OUTPUT_ANALYSIS_PATH)
1658     print("图像文件夹: ", OUTPUT_FIG_DIR)
1659
1660
1661 if __name__ == "__main__":
1662     main()

```